```
class InventoryRules {
  static InventoryState add(
    InventoryState state, {
    required String id,
    required String photoPath,
    required StorageZone zone,
    required DateTime now,
  }) {
    if (id.isEmpty ||
        photoPath.isEmpty ||
        state.foods.containsKey(id) ||
        state.reminders.any(
          (r) => r.references.any((ref) => ref.foodId == id),
        )) {
      throw ArgumentError('INVENTORY_ID_OR_PHOTO_INVALID');
    }
    final days = state.defaults[zone]!;
    final food = FoodRecord(
      id: id,
      photoPath: photoPath,
      zone: zone,
      addedAt: now,
      intervalDays: days,
      dueAt: scheduleCheck(
        now,
        days,
        hour: state.reminderHour,
        minute: state.reminderMinute,
      ),
    );
    return state.copyWith(
      foods: {...state.foods, id: food},
      nextNumber: state.nextNumber + 1,
    );
  }
  static FoodRecord _require(InventoryState state, String id) =>
      state.foods[id] ?? (throw StateError('INVENTORY_RECORD_MISSING'));
  static InventoryState organize(
    InventoryState state,
    Map<String, Remaining> selected,
    DateTime now,
  ) {
    if (selected.isEmpty) return state;
    final foods = {...state.foods};
    for (final entry in selected.entries) {
      final food = _require(state, entry.key);
      if (!food.active) throw StateError('INVENTORY_RECORD_ARCHIVED');
      foods[entry.key] = food.copyWith(
        remaining: entry.value,
        confirmedAt: now,
        cycle: food.cycle + 1,
        dueAt: scheduleCheck(
          now,
          food.intervalDays,
          hour: state.reminderHour,
          minute: state.reminderMinute,
        ),
```

```
      );
    }
    return state.copyWith(foods: foods, lastOrganizedAt: now);
  }
  static InventoryState edit(
    InventoryState state,
    String id, {
    StorageZone? zone,
    int? intervalDays,
    DateTime? dueAt,
    DateTime? addedAt,
    String? photoPath,
  }) {
    final food = _require(state, id);
    if (intervalDays != null && (intervalDays < 1 || intervalDays > 3650))
      throw ArgumentError('CHECK_INTERVAL_INVALID');
    if (photoPath != null && photoPath.isEmpty)
      throw ArgumentError('PHOTO_PATH_INVALID');
    return state.copyWith(
      foods: {
        ...state.foods,
        id: food.copyWith(
          zone: zone,
          intervalDays: intervalDays,
          dueAt: dueAt,
          addedAt: addedAt,
          photoPath: photoPath,
          cycle: dueAt == null ? food.cycle : food.cycle + 1,
        ),
      },
    );
  }
  static InventoryState restore(InventoryState state, String id, DateTime now) {
    final food = _require(state, id);
    if (food.active) throw StateError('INVENTORY_RECORD_NOT_ARCHIVED');
    return state.copyWith(
      foods: {
        ...state.foods,
        id: food.copyWith(
          remaining: Remaining.full,
          confirmedAt: now,
          cycle: food.cycle + 1,
          dueAt: scheduleCheck(
            now,
            food.intervalDays,
            hour: state.reminderHour,
            minute: state.reminderMinute,
          ),
        ),
      },
    );
  }
  static InventoryState delete(InventoryState state, String id) {
    _require(state, id);
    return state.copyWith(foods: {...state.foods}..remove(id));
  }
  static InventoryState changeDefaults(
```

```
    InventoryState state,
    Map<StorageZone, int> values, {
    int? hour,
    int? minute,
  }) {
    if (values.values.any((e) => e < 1 || e > 3650) ||
        (hour != null && (hour < 0 || hour > 23)) ||
        (minute != null && (minute < 0 || minute > 59))) {
      throw ArgumentError('CHECK_DEFAULTS_INVALID');
    }
    return state.copyWith(
      defaults: {...state.defaults, ...values},
      reminderHour: hour,
      reminderMinute: minute,
    );
  }
  static InventoryState collectDue(InventoryState state, DateTime now) {
    final byDay = {
      for (final r in state.reminders) r.day: [...r.references],
    };
    for (final food in state.active) {
      if (food.dueAt.isAfter(now) ||
          byDay.values.any((refs) => refs.any((ref) => ref.matches(food))))
        continue;
      byDay
          .putIfAbsent(calendarDay(food.dueAt), () => [])
          .add(CycleReference(food.id, food.cycle));
    }
    final days = byDay.keys.toList()..sort((a, b) => b.compareTo(a));
    return state.copyWith(
      reminders: days.map((d) => CheckReminder(d, byDay[d]!)),
    );
  }
}
class CycleReference {
  final String foodId;
  final int cycle;
  const CycleReference(this.foodId, this.cycle);
  bool matches(FoodRecord food) => foodId == food.id && cycle == food.cycle;
  Map<String, dynamic> toJson() => {'foodId': foodId, 'cycle': cycle};
  factory CycleReference.fromJson(Map<String, dynamic> json) =>
      CycleReference(json['foodId'] as String, json['cycle'] as int);
}
class CheckReminder {
  final String day;
  final List<CycleReference> references;
  CheckReminder(this.day, Iterable<CycleReference> references)
    : references = List.unmodifiable(references);
  int get originalCount => references.length;
  List<FoodRecord> pending(InventoryState state, DateTime now) => [
    for (final ref in references)
      if (state.foods[ref.foodId] case final food?)
        if (food.active && ref.matches(food) && !food.dueAt.isAfter(now)) food,
  ];
  Map<String, dynamic> toJson() => {
    'day': day,
    'references': references.map((e) => e.toJson()).toList(),
```

```
  };
  factory CheckReminder.fromJson(Map<String, dynamic> json) => CheckReminder(
    json['day'] as String,
    (json['references'] as List).map(
      (e) => CycleReference.fromJson(Map<String, dynamic>.from(e as Map)),
    ),
  );
}
class InventoryState {
  final Map<String, FoodRecord> foods;
  final List<CheckReminder> reminders;
  final Map<StorageZone, int> defaults;
  final DateTime? lastOrganizedAt;
  final int reminderHour;
  final int reminderMinute;
  final int nextNumber;
  InventoryState({
    Map<String, FoodRecord> foods = const {},
    Iterable<CheckReminder> reminders = const [],
    Map<StorageZone, int> defaults = const {
      StorageZone.cold: 3,
      StorageZone.frozen: 14,
      StorageZone.pantry: 3,
    },
    this.lastOrganizedAt,
    this.reminderHour = 17,
    this.reminderMinute = 30,
    this.nextNumber = 1,
  }) : foods = Map.unmodifiable(foods),
       reminders = List.unmodifiable(reminders),
       defaults = Map.unmodifiable(defaults);
  List<FoodRecord> get active => foods.values.where((e) => e.active).toList();
  List<FoodRecord> get history => foods.values.where((e) => !e.active).toList();
  bool hasPending(DateTime now) =>
      reminders.any((r) => r.pending(this, now).isNotEmpty);
  bool shouldShowInactivity(DateTime now) {
    if (active.isEmpty || active.any((food) => !food.dueAt.isAfter(now)))
      return false;
    final origin =
        lastOrganizedAt ??
        active.map((f) => f.addedAt).reduce((a, b) => a.isBefore(b) ? a : b);
    return now.difference(origin) > const Duration(days: 2);
  }
  InventoryState copyWith({
    Map<String, FoodRecord>? foods,
    Iterable<CheckReminder>? reminders,
    Map<StorageZone, int>? defaults,
    DateTime? lastOrganizedAt,
    int? reminderHour,
    int? reminderMinute,
    int? nextNumber,
  }) => InventoryState(
    foods: foods ?? this.foods,
    reminders: reminders ?? this.reminders,
    defaults: defaults ?? this.defaults,
    lastOrganizedAt: lastOrganizedAt ?? this.lastOrganizedAt,
    reminderHour: reminderHour ?? this.reminderHour,
```

```
    reminderMinute: reminderMinute ?? this.reminderMinute,
    nextNumber: nextNumber ?? this.nextNumber,
  );
  Map<String, dynamic> toJson() => {
    'schemaVersion': 1,
    'nextNumber': nextNumber,
    'foods': foods.values.map((e) => e.toJson()).toList(),
    'reminders': reminders.map((e) => e.toJson()).toList(),
    'defaults': {
      for (final entry in defaults.entries) entry.key.name: entry.value,
    },
    'lastOrganizedAt': lastOrganizedAt?.toIso8601String(),
    'reminderHour': reminderHour,
    'reminderMinute': reminderMinute,
  };
  factory InventoryState.fromJson(Map<String, dynamic> json) {
    if (json['schemaVersion'] != 1)
      throw const FormatException('INVENTORY_SCHEMA_UNSUPPORTED');
    final records = (json['foods'] as List)
        .map((e) => FoodRecord.fromJson(Map<String, dynamic>.from(e as Map)))
        .toList();
    if (records.map((e) => e.id).toSet().length != records.length)
      throw const FormatException('INVENTORY_DUPLICATE_ID');
    final defaults = {
      for (final entry in (json['defaults'] as Map).entries)
        StorageZone.values.byName(entry.key as String): entry.value as int,
    };
    final hour = json['reminderHour'] as int;
    final minute = json['reminderMinute'] as int;
    final nextNumber =
        json['nextNumber'] as int? ??
        records.fold<int>(1, (next, food) {
          final number = int.tryParse(food.id);
          return number != null && number >= next ? number + 1 : next;
        });
    if (defaults.length != StorageZone.values.length ||
        defaults.values.any((d) => d < 1 || d > 3650) ||
        hour < 0 ||
        hour > 23 ||
        minute < 0 ||
        minute > 59 ||
        nextNumber < 1)
      throw const FormatException('INVENTORY_DEFAULTS_INVALID');
    return InventoryState(
      foods: {for (final f in records) f.id: f},
      reminders: (json['reminders'] as List).map(
        (e) => CheckReminder.fromJson(Map<String, dynamic>.from(e as Map)),
      ),
      defaults: defaults,
      lastOrganizedAt: json['lastOrganizedAt'] == null
          ? null
          : DateTime.parse(json['lastOrganizedAt'] as String),
      reminderHour: hour,
      reminderMinute: minute,
      nextNumber: nextNumber,
    );
  }
```

```
}
abstract interface class InventoryStorage {
  Future<InventoryState?> load();
  Future<void> save(InventoryState state);
}
class InventoryRepository {
  final InventoryStorage storage;
  final DateTime Function() clock;
  InventoryState _state = InventoryState();
  InventoryState? _undo;
  Future<void> _tail = Future.value();
  bool _loaded = false;
  int _revision = 0;
  final _changes = StreamController<InventoryState>.broadcast(sync: true);
  InventoryRepository(this.storage, {DateTime Function()? clock})
    : clock = clock ?? DateTime.now;
  InventoryState get state => _state;
  bool get canUndo => _undo != null;
  int get revision => _revision;
  Set<String> get retainedPhotoPaths => {
    for (final food in _state.foods.values) food.photoPath,
    if (_undo != null)
      for (final food in _undo!.foods.values) food.photoPath,
  };
  Stream<InventoryState> get changes => _changes.stream;
  Future<void> _serialize(Future<void> Function() action) {
    final operation = _tail.then((_) => action());
    _tail = operation.catchError((Object _) {});
    return operation;
  }
  Future<void> open() => _serialize(() async {
    if (_loaded) return;
    final loaded = await storage.load() ?? InventoryState();
    final current = InventoryRules.collectDue(loaded, clock());
    await storage.save(current);
    _state = current;
    _loaded = true;
    _revision++;
    _changes.add(_state);
  });
  Future<void> update(
    InventoryState Function(InventoryState) operation, {
    bool undoable = true,
    bool invalidateUndo = false,
  }) => _serialize(() async {
    if (!_loaded) throw StateError('INVENTORY_NOT_OPEN');
    final next = InventoryRules.collectDue(operation(_state), clock());
    await storage.save(next);
    if (undoable)
      _undo = _state;
    else if (invalidateUndo)
      _undo = null;
    _state = next;
    _revision++;
    _changes.add(_state);
  });
  Future<void> refreshDue() => _serialize(() async {
```

```
    if (!_loaded) throw StateError('INVENTORY_NOT_OPEN');
    final next = InventoryRules.collectDue(_state, clock());
    if (jsonEncode(next.toJson()) == jsonEncode(_state.toJson())) return;
    await storage.save(next);
    _state = next;
    _undo = null;
    _revision++;
    _changes.add(_state);
  });
  Future<void> undo({int? expectedRevision}) => _serialize(() async {
    if (expectedRevision != null && expectedRevision != _revision) {
      throw StateError('UNDO_STATE_CHANGED');
    }
    final previous = _undo;
    if (previous == null) return;
    final next = InventoryRules.collectDue(
      previous.copyWith(
        nextNumber: previous.nextNumber > _state.nextNumber
            ? previous.nextNumber
            : _state.nextNumber,
      ),
      clock(),
    );
    await storage.save(next);
    _state = next;
    _undo = null;
    _revision++;
    _changes.add(_state);
  });
  Future<void> close() async {
    await _tail;
    await _changes.close();
  }
}
class FileInventoryStorage implements InventoryStorage {
  final Directory directory;
  bool recoveredFromBackup = false;
  FileInventoryStorage(this.directory);
  File get _main => File('${directory.path}/inventory.json');
  File get _backup => File('${directory.path}/inventory.backup.json');
  File get _temporary => File('${directory.path}/inventory.pending.json');
  File get _backupTemporary =>
      File('${directory.path}/inventory.backup.pending.json');
  Future<InventoryState> _decode(File file) async => InventoryState.fromJson(
    jsonDecode(await file.readAsString()) as Map<String, dynamic>,
  );
  Future<Set<String>> photoReferences() async {
    final paths = <String>{};
    for (final file in [_main, _backup]) {
      if (await file.exists()) {
        paths.addAll(
          (await _decode(file)).foods.values.map((f) => f.photoPath),
        );
      }
    }
    return paths;
  }
```

```
  @override
  Future<InventoryState?> load() async {
    if (await _main.exists()) {
      try {
        return await _decode(_main);
      } on FormatException {
        return _recover();
      } on TypeError {
        return _recover();
      } on ArgumentError {
        return _recover();
      }
    }
    if (await _backup.exists()) return _recover();
    return null;
  }
  Future<InventoryState> _recover() async {
    if (!await _backup.exists())
      throw const FormatException('INVENTORY_RECOVERY_REQUIRED');
    final saved = await _decode(_backup);
    recoveredFromBackup = true;
    return saved;
  }
  @override
  Future<void> save(InventoryState state) async {
    await directory.create(recursive: true);
    await _temporary.writeAsString(jsonEncode(state.toJson()), flush: true);
    final previousFile = recoveredFromBackup ? _backup : _main;
    if (await previousFile.exists()) {
      final previous = await _decode(previousFile);
      final recoverable = previous.copyWith(
        foods: {
          for (final entry in previous.foods.entries)
            if (state.foods.containsKey(entry.key)) entry.key: entry.value,
        },
        nextNumber: previous.nextNumber > state.nextNumber
            ? previous.nextNumber
            : state.nextNumber,
      );
      await _backupTemporary.writeAsString(
        jsonEncode(recoverable.toJson()),
        flush: true,
      );
      await _backupTemporary.rename(_backup.path);
    }
    if (recoveredFromBackup) {
      if (await _main.exists()) {
        await _main.copy(
          '${directory.path}/inventory.corrupt.${DateTime.now().microsecondsSinceEpoch}.json',
        );
      }
    }
    try {
      await _temporary.rename(_main.path);
      recoveredFromBackup = false;
    } on FileSystemException {
      if (!await _main.exists() && await _backup.exists())
```

```
        await _backup.copy(_main.path);
      rethrow;
    }
  }
}
part of 'bootstrap.dart';
mixin _InventoryActions {
  InventoryRepository? get inventory;
  Directory? get dataDirectory;
  void recordEvent(String name, Map<String, String> properties);
  Future<void> organize(FoodRecord food, Remaining remaining) async {
    await inventory!.update(
      (state) =>
          InventoryRules.organize(state, {food.id: remaining}, DateTime.now()),
    );
    recordEvent('fridge.review.updated.v1', {
      'state': remaining.name,
      'zone': (inventory!.state.foods[food.id] ?? food).zone.name,
    });
  }
  Future<void> restore(FoodRecord food) => inventory!.update(
    (state) => InventoryRules.restore(state, food.id, DateTime.now()),
  );
  Future<void> editRecord(
    FoodRecord food, {
    required StorageZone zone,
    required int intervalDays,
    required DateTime addedAt,
    DateTime? dueAt,
  }) => inventory!.update(
    (state) => InventoryRules.edit(
      state,
      food.id,
      zone: zone,
      intervalDays: intervalDays,
      addedAt: addedAt,
      dueAt: dueAt,
    ),
  );
  PhotoLifecycle _photoLifecycle() => PhotoLifecycle(
    repository: inventory!,
    photos: ManagedFoodPhotos(directory: dataDirectory!),
    snapshotReferences: FileInventoryStorage(dataDirectory!).photoReferences,
  );
  Future<void> replacePhoto(FoodRecord food) =>
      _photoLifecycle().replace(food.id);
  Future<void> deleteRecord(FoodRecord food) async {
    await inventory!.update(
      (state) => InventoryRules.delete(state, food.id),
      undoable: false,
      invalidateUndo: true,
    );
    try {
      await ManagedFoodPhotos(
        directory: dataDirectory!,
      ).discardSaved(food.photoPath);
    } catch (_) {
```

```
      /* A failed private-file cleanup cannot resurrect a deleted record. */
    }
  }
}
abstract interface class FoodPhotoSource {
  Future<String> capture();
  Future<List<String>> album();
  Future<String> persist(String path);
  Future<void> discardSaved(String path);
}
class PendingPhoto {
  final String path;
  final StorageZone zone;
  final String origin;
  String? savedPath;
  PendingPhoto(this.path, this.zone, this.origin);
}
class CaptureFlow extends ChangeNotifier {
  final InventoryRepository repository;
  final FoodPhotoSource source;
  final DateTime Function() clock;
  final ProductEvent? onEvent;
  StorageZone zone;
  bool busy = false;
  bool failed = false;
  int savedCount = 0;
  final List<PendingPhoto> _pending = [];
  bool _closed = false;
  bool get hasPending => _pending.isNotEmpty;
  CaptureFlow({
    required this.repository,
    required this.source,
    required this.zone,
    DateTime Function()? clock,
    this.onEvent,
  }) : clock = clock ?? DateTime.now;
  void selectZone(StorageZone value) {
    zone = value;
    _publish();
  }
  Future<void> take() => _run(() async {
    if (_pending.isNotEmpty) return;
    final destination = zone;
    _event('fridge.capture.started.v1', 'camera', destination);
    _pending.add(PendingPhoto(await source.capture(), destination, 'camera'));
  }, origin: 'camera');
  Future<void> chooseAlbum() => _run(() async {
    if (_pending.isNotEmpty) return;
    final destination = zone;
    _event('fridge.capture.started.v1', 'album', destination);
    _pending.addAll(
      (await source.album()).map(
        (path) => PendingPhoto(path, destination, 'album'),
      ),
    );
  }, origin: 'album');
  Future<void> retry() => _run(() async {});
```

```
  Future<void> abandon() async {
    if (busy) return;
    for (final photo in _pending) {
      if (photo.savedPath != null) {
        try {
          await source.discardSaved(photo.savedPath!);
        } catch (_) {
          /* Unreferenced private files can be cleaned later. */
        }
      }
    }
    _pending.clear();
    failed = false;
    _publish();
  }
  Future<void> _run(Future<void> Function() acquire, {String? origin}) async {
    if (busy || _closed) return;
    busy = true;
    failed = false;
    _publish();
    final destination = zone;
    var acquiring = true;
    try {
      await acquire();
      acquiring = false;
      while (_pending.isNotEmpty) {
        final photo = _pending.first;
        photo.savedPath ??= await source.persist(photo.path);
        await repository.update(
          (state) => InventoryRules.add(
            state,
            id: state.nextNumber.toString().padLeft(3, '0'),
            photoPath: photo.savedPath!,
            zone: photo.zone,
            now: clock(),
          ),
        );
        _pending.removeAt(0);
        savedCount++;
        _event(
          'fridge.inventory.added.v1',
          photo.origin,
          photo.zone,
          'success',
        );
      }
    } catch (_) {
      failed = true;
      final photo = _pending.firstOrNull;
      if (photo != null || origin != null) {
        _event(
          'fridge.inventory.added.v1',
          photo?.origin ?? origin!,
          photo?.zone ?? destination,
          acquiring ? 'read_failed' : 'save_failed',
        );
      }
```

```
    } finally {
      busy = false;
      _publish();
    }
  }
  void _event(String name, String origin, StorageZone zone, [String? result]) {
    if (_closed) return;
    emitProductEvent(onEvent, name, {
      'source': origin,
      'zone': zone.name,
      'result': ?result,
    });
  }
  void _publish() {
    if (!_closed) notifyListeners();
  }
  @override
  void dispose() {
    _closed = true;
    super.dispose();
  }
}
abstract interface class PhotoLibrary {
  Future<String?> replacement();
  Future<void> discardSaved(String path);
  Future<void> collectUnreferenced(Set<String> retained);
}
class PhotoLifecycle {
  final InventoryRepository repository;
  final PhotoLibrary photos;
  final Future<Set<String>> Function() snapshotReferences;
  PhotoLifecycle({
    required this.repository,
    required this.photos,
    required this.snapshotReferences,
  });
  Future<Set<String>> _retained() async => {
    ...await snapshotReferences(),
    ...repository.retainedPhotoPaths,
  };
  Future<bool> recoverOrphans() async {
    try {
      await photos.collectUnreferenced(await _retained());
      return true;
    } catch (_) {
      return false;
    }
  }
  Future<void> replace(String id) async {
    final candidate = await photos.replacement();
    if (candidate == null) return;
    try {
      await repository.update(
        (state) => InventoryRules.edit(state, id, photoPath: candidate),
      );
    } catch (_) {
      try {
```

```
        if (!(await _retained()).contains(candidate)) {
          await photos.discardSaved(candidate);
        }
      } catch (_) {
      }
      rethrow;
    }
  }
}
class NotificationStatus {
  final bool enabled, authorized, requested, limited;
  const NotificationStatus({
    this.enabled = false,
    this.authorized = false,
    this.requested = false,
    this.limited = false,
  });
  factory NotificationStatus.fromJson(Map<String, dynamic> value) =>
      NotificationStatus(
        enabled: value['enabled'] == true,
        authorized: value['authorized'] == true,
        requested: value['requested'] == true,
        limited: value['limited'] == true,
      );
}
abstract interface class LocalNotificationPlatform {
  Stream<String> get taps;
  Future<NotificationStatus> status();
  Future<NotificationStatus> request();
  Future<NotificationStatus> setEnabled(bool value);
  Future<void> sync(List<Map<String, Object>> cycles, Map<String, String> copy);
  Future<String?> consumeTap();
  Future<void> openSettings();
  Future<void> close();
}
class NotificationSession extends ChangeNotifier {
  final InventoryRepository repository;
  final LocalNotificationPlatform platform;
  final bool Function() consent;
  final Map<String, String> Function() copy;
  final DateTime Function() clock;
  final ProductEvent? onEvent;
  NotificationStatus status = const NotificationStatus();
  String? errorKey, pendingDay;
  bool busy = false, _closed = false, _active = true, _queued = false;
  Future<void>? _tail;
  StreamSubscription<InventoryState>? _inventory;
  StreamSubscription<String>? _taps;
  Timer? _timer;
  NotificationSession({
    required this.repository,
    required this.platform,
    required this.consent,
    required this.copy,
    DateTime Function()? clock,
    this.onEvent,
  }) : clock = clock ?? DateTime.now;
```

```
  Future<void> start() async {
    if (_closed || _inventory != null || !consent()) return;
    _inventory = repository.changes.listen((_) {
      unawaited(synchronize());
      _scheduleClock();
    });
    _taps = platform.taps.listen(_tap);
    await synchronize();
    await _drainTap();
    _scheduleClock();
  }
  void _tap(String day) {
    if (_closed ||
        !consent() ||
        !RegExp(r'^\d{4}-\d{2}-\d{2}$').hasMatch(day)) {
      return;
    }
    pendingDay = day;
    _notify();
  }
  String? takeTap() {
    final day = pendingDay;
    pendingDay = null;
    return day;
  }
  Future<void> _drainTap() async {
    try {
      final day = await platform.consumeTap();
      if (day != null) _tap(day);
    } catch (_) {}
  }
  Future<void> _serialize(Future<void> Function() operation) {
    late final Future<void> result;
    result = (_tail ?? Future<void>.value())
        .then((_) async {
          if (_closed || !consent()) return;
          try {
            await operation();
            errorKey = null;
          } catch (_) {
            errorKey = 'notificationUnavailable';
          }
          _notify();
        })
        .whenComplete(() {
          if (identical(_tail, result)) _tail = null;
        });
    _tail = result;
    return result;
  }
  Future<void> synchronize() {
    if (_closed || _queued || !consent()) return _tail ?? Future.value();
    _queued = true;
    return _serialize(() async {
      _queued = false;
      status = await platform.status();
      await platform.sync(notificationPlan(repository.state), copy());
```

```
      status = await platform.status();
    }).whenComplete(() {
      _queued = false;
    });
  }
  Future<void> enable(Future<bool> Function() explain) async {
    if (busy || _closed || !consent()) return;
    busy = true;
    errorKey = null;
    _notify();
    await _serialize(() async {
      status = await platform.status();
      if (!status.authorized) {
        if (!await explain()) {
          _permissionEvent('cancelled');
          return;
        }
        if (_closed || !consent()) return;
        status = await platform.request();
        _permissionEvent(status.authorized ? 'granted' : 'denied');
      }
      if (_closed || !consent() || !status.authorized) return;
      status = await platform.setEnabled(true);
      await platform.sync(notificationPlan(repository.state), copy());
      status = await platform.status();
    });
    busy = false;
    _notify();
  }
  Future<void> disable() async {
    if (busy || _closed) return;
    busy = true;
    _notify();
    await _serialize(() async {
      status = await platform.setEnabled(false);
    });
    busy = false;
    _notify();
  }
  void _permissionEvent(String result) {
    if (!_closed && consent()) {
      emitProductEvent(onEvent, 'fridge.permission.resolved.v1', {
        'permission': 'notifications',
        'result': result,
      });
    }
  }
  Future<void> openSettings() => _serialize(platform.openSettings);
  void setActive(bool value) {
    _active = value;
    _timer?.cancel();
    if (value && !_closed && consent()) unawaited(refresh());
  }
  Future<void> refresh() async {
    if (_closed || !consent()) return;
    try {
      await repository.refreshDue();
```

```
      await synchronize();
      await _drainTap();
    } catch (_) {
      errorKey = 'error';
      _notify();
    }
    _scheduleClock();
  }
  void _scheduleClock() {
    _timer?.cancel();
    if (!_active || _closed || !consent()) return;
    var wait = const Duration(seconds: 30);
    final now = clock();
    for (final food in repository.state.active) {
      final delay = food.dueAt.difference(now);
      if (delay > Duration.zero && delay < wait) wait = delay;
    }
    _timer = Timer(wait, () => unawaited(refresh()));
  }
  void _notify() {
    if (!_closed) notifyListeners();
  }
  Future<void> close() async {
    if (_closed) return;
    _closed = true;
    _timer?.cancel();
    await _inventory?.cancel();
    await _taps?.cancel();
    await _tail;
    await platform.close();
  }
  @override
  void dispose() {
    unawaited(close());
    super.dispose();
  }
}
List<FoodRecord> reviewItems(
  InventoryState state,
  DateTime now, {
  StorageZone? zone,
  String? reminderDay,
}) {
  final foods = reminderDay == null
      ? state.active.where((food) => zone == null || food.zone == zone).toList()
      : state.reminders
                .where((reminder) => reminder.day == reminderDay)
                .firstOrNull
                ?.pending(state, now) ??
            <FoodRecord>[];
  return foods..sort((a, b) {
    final due = a.dueAt.compareTo(b.dueAt);
    return due == 0 ? a.id.compareTo(b.id) : due;
  });
}
class CameraCoordinator {
  late final CameraSession camera;
```

```
  late final CaptureFlow flow;
  late final FenlengPermissionOrchestrator permissions;
  bool _wantCamera = true;
  CameraCoordinator({
    required Directory directory,
    required InventoryRepository repository,
    required StorageZone zone,
    required FridgeStrings strings,
    required FenlengPermissionRationalePresenter rationale,
    ProductEvent? onEvent,
  }) {
    permissions = FenlengPermissionOrchestrator(
      channel: FridgeBootstrap.channel == 'china'
          ? FenlengReleaseChannel.china
          : FenlengReleaseChannel.global,
      platform: const PermissionHandlerFenlengAdapter(),
      presentRationale: rationale,
      purposes: [
        FenlengPermissionPurpose(
          featureId: 'food.capture',
          permission: FenlengPermission.camera,
          featureName: strings.text('add'),
          dataAccessed: strings.text('localOnly'),
          purpose: strings.text('permissionBody'),
          denialImpact: strings.text('cameraUnavailable'),
          policyUrl: Uri.parse('https://fridge.fenlengtech.com/privacy/'),
        ),
      ],
    );
    camera = CameraSession(() async {
      if (!_wantCamera) return false;
      final previous = await permissions.platform.status(
        FenlengPermission.camera,
      );
      final outcome = await permissions.requestForFeature('food.capture');
      if (previous != FenlengPermissionStatus.granted) {
        emitProductEvent(onEvent, 'fridge.permission.resolved.v1', {
          'permission': 'camera',
          'result': switch (outcome) {
            FenlengPermissionOutcome.granted => 'granted',
            FenlengPermissionOutcome.cancelled => 'cancelled',
            FenlengPermissionOutcome.denied => 'denied',
            _ => 'restricted',
          },
        });
      }
      final granted = outcome == FenlengPermissionOutcome.granted;
      if (!granted) _wantCamera = false;
      return granted;
    });
    flow = CaptureFlow(
      repository: repository,
      zone: zone,
      source: ManagedFoodPhotos(directory: directory, camera: camera),
      onEvent: onEvent,
    );
  }
```

```
  Listenable get changes => Listenable.merge([camera, flow]);
  Future<void> open() {
    _wantCamera = true;
    return camera.open();
  }
  Future<void> resume() async {
    if (_wantCamera) await camera.open();
  }
  Future<void> chooseAlbum() {
    if (camera.controller?.value.isInitialized != true) _wantCamera = false;
    return flow.chooseAlbum();
  }
  Future<void> openSettings() async {
    await permissions.openSettings();
  }
  Future<void> close() async {
    flow.dispose();
    await camera.close();
  }
}
part 'cabinet_door.dart';
part 'cabinet_photos.dart';
class CabinetScreen extends StatelessWidget {
  final InventoryState state;
  final FridgeStrings strings;
  final ImageProvider Function(String) photo;
  final ValueChanged<StorageZone> onOpenZone;
  final VoidCallback onSettings, onReminders, onReview;
  const CabinetScreen({
    super.key,
    required this.state,
    required this.strings,
    required this.photo,
    required this.onOpenZone,
    required this.onSettings,
    required this.onReminders,
    required this.onReview,
  });
  @override
  Widget build(BuildContext context) => FridgePage(
    title: strings.text('home'),
    back: false,
    actions: [
      FridgeHeaderIcon(
        label: strings.text('notifications'),
        onPressed: onReminders,
        icon: Builder(
          builder: (context) => Badge(
            isLabelVisible: state.hasPending(DateTime.now()),
            child: FridgeIcon(
              FridgeIconName.bell,
              scale: FridgeLayout.of(context).scale,
            ),
          ),
        ),
      ),
      FridgeHeaderIcon(
```

```
        label: strings.text('settings'),
        onPressed: onSettings,
        icon: Builder(
          builder: (context) => FridgeIcon(
            FridgeIconName.settings,
            scale: FridgeLayout.of(context).scale,
          ),
        ),
      ),
    ],
    child: Builder(
      builder: (context) {
        final layout = FridgeLayout.of(context);
        final palette = Theme.of(context).extension<FridgePalette>()!;
        final now = DateTime.now();
        final hint = state.active.any((food) => !food.dueAt.isAfter(now))
            ? 'dueHint'
            : state.shouldShowInactivity(now)
            ? 'inactivityHint'
            : null;
        return ListView(
          padding: EdgeInsets.only(
            top: layout.d(FridgeProductTokens.spaceCabinetTop),
            bottom: layout.d(FridgeProductTokens.spaceBottom),
          ),
          children: [
            Stack(
              clipBehavior: Clip.none,
              children: [
                PositionedDirectional(
                  start: layout.d(FridgeProductTokens.spaceFeetInset),
                  end: layout.d(FridgeProductTokens.spaceFeetInset),
                  bottom: layout.d(-FridgeProductTokens.sizeFeetHeight),
                  child: Row(
                    mainAxisAlignment: MainAxisAlignment.spaceBetween,
                    children: [
                      for (var i = 0; i < 2; i++)
                        Container(
                          width: layout.d(FridgeProductTokens.sizeFeetWidth),
                          height: layout.d(FridgeProductTokens.sizeFeetHeight),
                          decoration: BoxDecoration(
                            color: palette.color('feet'),
                            borderRadius: BorderRadius.vertical(
                              bottom: Radius.circular(layout.d(2)),
                            ),
                          ),
                        ),
                    ],
                  ),
                ),
                Container(
                  key: const ValueKey('cabinet'),
                  padding: EdgeInsets.all(
                    layout.d(FridgeProductTokens.spaceCabinet),
                  ),
                  decoration: BoxDecoration(
                    color: palette.color('cabinet'),
```

```
      border: Border.all(
        color: palette.color('cabinetBorder'),
        width: layout.d(FridgeProductTokens.sizeBorder),
      ),
      borderRadius: BorderRadius.vertical(
        top: Radius.circular(
          layout.d(FridgeProductTokens.radiusCabinetTop),
        ),
        bottom: Radius.circular(
          layout.d(FridgeProductTokens.radiusCabinetBottom),
        ),
      ),
      boxShadow: [
        BoxShadow(
          color: palette.color('cabinetShadow'),
          offset: Offset(0, layout.d(8)),
          blurRadius: layout.d(15),
        ),
      ],
    ),
    child: LayoutBuilder(
      builder: (context, box) => Column(
        children: [
          _door(context, StorageZone.cold, box.maxWidth),
          SizedBox(
            height: layout.d(FridgeProductTokens.gapCabinet),
          ),
          IntrinsicHeight(
            child: Row(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                Expanded(
                  child: _door(
                    context,
                    StorageZone.frozen,
                    (box.maxWidth -
                            layout.d(
                              FridgeProductTokens.gapCabinet,
                            )) /
                        2,
                  ),
                ),
                SizedBox(
                  width: layout.d(FridgeProductTokens.gapCabinet),
                ),
                Expanded(
                  child: _door(
                    context,
                    StorageZone.pantry,
                    (box.maxWidth -
                            layout.d(
                              FridgeProductTokens.gapCabinet,
                            )) /
                        2,
                  ),
                ),
              ],
```

```
                          ),
                        ),
                      ],
                    ),
                  ),
                ),
              ],
            ),
            if (hint != null)
              Padding(
                padding: EdgeInsets.only(
                  top: layout.d(FridgeProductTokens.spaceSection),
                ),
                child: Container(
                  padding: EdgeInsets.all(
                    layout.d(FridgeProductTokens.spaceCard),
                  ),
                  decoration: BoxDecoration(
                    color: palette.color('surface'),
                    borderRadius: BorderRadius.circular(
                      layout.d(FridgeProductTokens.radiusCard),
                    ),
                    border: Border.all(
                      color: palette.border,
                      width: layout.d(FridgeProductTokens.sizeBorder),
                    ),
                  ),
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Text(strings.text(hint)),
                      TextButton(
                        onPressed: onReview,
                        child: Text(strings.text('goOrganize')),
                      ),
                    ],
                  ),
                ),
              ),
          ],
        );
      },
    ),
  );
  Widget _door(BuildContext context, StorageZone zone, double width) =>
      _CabinetDoor(
        foods: state.active.where((food) => food.zone == zone).toList(),
        strings: strings,
        photo: photo,
        zone: zone,
        width: width,
        onTap: () => onOpenZone(zone),
      );
}
part of 'cabinet.dart';
class _CabinetDoor extends StatelessWidget {
  final List<FoodRecord> foods;
```

```
  final FridgeStrings strings;
  final ImageProvider Function(String) photo;
  final StorageZone zone;
  final double width;
  final VoidCallback onTap;
  const _CabinetDoor({
    required this.foods,
    required this.strings,
    required this.photo,
    required this.zone,
    required this.width,
    required this.onTap,
  });
  @override
  Widget build(BuildContext context) {
    final layout = FridgeLayout.of(context);
    final palette = Theme.of(context).extension<FridgePalette>()!;
    final tall = zone == StorageZone.cold;
    final radius = BorderRadius.only(
      topLeft: Radius.circular(
        layout.d(
          tall
              ? FridgeProductTokens.radiusColdTop
              : FridgeProductTokens.radiusDoor,
        ),
      ),
      topRight: Radius.circular(
        layout.d(
          tall
              ? FridgeProductTokens.radiusColdTop
              : FridgeProductTokens.radiusDoor,
        ),
      ),
      bottomLeft: Radius.circular(
        layout.d(
          zone == StorageZone.frozen
              ? FridgeProductTokens.radiusOuter
              : FridgeProductTokens.radiusDoor,
        ),
      ),
      bottomRight: Radius.circular(
        layout.d(
          zone == StorageZone.pantry
              ? FridgeProductTokens.radiusOuter
              : FridgeProductTokens.radiusDoor,
        ),
      ),
    );
    return Semantics(
      button: true,
      label:
          '${strings.text(zone.name)} ${foods.isEmpty ? strings.text('empty') :
strings.format('recordCount', {'count': foods.length})}',
      child: Material(
        color: palette.color(zone.name),
        borderRadius: radius,
        child: InkWell(
```

```
          onTap: onTap,
          borderRadius: radius,
          child: Container(
            key: ValueKey('door-${zone.name}'),
            constraints: BoxConstraints(
              minHeight: layout.d(
                tall
                    ? FridgeProductTokens.sizeCold
                    : FridgeProductTokens.sizeLower,
              ),
            ),
            decoration: BoxDecoration(
              border: Border.all(
                color: palette.color('${zone.name}Line'),
                width: layout.d(FridgeProductTokens.sizeBorder),
              ),
              borderRadius: radius,
            ),
            child: Stack(
              children: [
                Padding(
                  padding: EdgeInsets.fromLTRB(
                    layout.d(FridgeProductTokens.spaceDoorSide),
                    layout.d(FridgeProductTokens.spaceDoorTop),
                    layout.d(FridgeProductTokens.spaceDoorSide),
                    layout.d(FridgeProductTokens.spaceDoorBottom),
                  ),
                  child: Column(
                    mainAxisSize: MainAxisSize.min,
                    children: [
                      Padding(
                        padding: EdgeInsets.only(
                          bottom: layout.d(
                            FridgeProductTokens.spaceDoorHeaderBottom,
                          ),
                        ),
                        child: Row(
                          crossAxisAlignment: CrossAxisAlignment.start,
                          children: [
                            Expanded(
                              child: Row(
                                crossAxisAlignment: CrossAxisAlignment.start,
                                children: [
                                  FridgeIcon(
                                    FridgeIconName.values.byName(zone.name),
                                    scale: layout.scale,
                                  ),
                                  SizedBox(
                                    width: layout.d(
                                      FridgeProductTokens.spaceLabelIcon,
                                    ),
                                  ),
                                  Expanded(
                                    child: Text(
                                      strings.text(zone.name),
                                      style: TextStyle(
                                        fontSize: layout.font(
```

```
                                  FridgeProductTokens.fontLabel,
                                ),
                                height: 1.23,
                                fontWeight: FontWeight.w400,
                              ),
                            ),
                          ),
                        ],
                      ),
                    ),
                    if (foods.isNotEmpty)
                      Padding(
                        padding: EdgeInsetsDirectional.only(
                          start: layout.d(4),
                        ),
                        child: Text(
                          strings.format('recordCountShort', {
                            'count': foods.length,
                          }),
                          style: TextStyle(
                            fontSize: layout.font(
                              FridgeProductTokens.fontMetadata,
                            ),
                            height: 1.45,
                            color: palette.color('count'),
                          ),
                        ),
                      ),
                  ],
                ),
              ),
              _photos(
                context,
                foods,
                width:
                    width -
                    layout.d(
                      2 *
                          (FridgeProductTokens.spaceDoorSide +
                              FridgeProductTokens.sizeBorder),
                    ),
                tall: tall,
              ),
              Padding(
                padding: EdgeInsets.only(
                  top: layout.d(FridgeProductTokens.spaceDoorFooterTop),
                ),
                child: Text(
                  '${strings.text(foods.isEmpty ? 'empty' : 'manage')}${foods.isEmpty ?
'' : '\u2002\u200A↗\uFE0E'}',
                  textAlign: TextAlign.center,
                  style: TextStyle(
                    fontSize: layout.font(
                      FridgeProductTokens.fontCaption,
                    ),
                    height: 1.2,
                    color: palette.color('doorFooter'),
```

```
                        ),
                      ),
                    ),
                  ],
                ),
              ),
              if (zone != StorageZone.pantry)
                PositionedDirectional(
                  end: layout.d(FridgeProductTokens.spaceHandleRight),
                  top: layout.d(FridgeProductTokens.spaceHandleTop),
                  child: Container(
                    width: layout.d(FridgeProductTokens.sizeHandleWidth),
                    height: layout.d(FridgeProductTokens.sizeHandleHeight),
                    decoration: BoxDecoration(
                      color: palette.color('handle'),
                      borderRadius: BorderRadius.circular(layout.d(5)),
                    ),
                  ),
                ),
            ],
          ),
        ),
      ),
    ),
  );
 }
}
part of 'cabinet.dart';
extension _CabinetPhotoLayout on _CabinetDoor {
 Widget _photos(
   BuildContext context,
   List<FoodRecord> foods, {
   required bool tall,
   required double width,
 }) {
   final layout = FridgeLayout.of(context);
   final palette = Theme.of(context).extension<FridgePalette>()!;
   final capacity = tall ? 6 : 3;
   final shown = foods
       .take(foods.length > capacity ? capacity - 1 : capacity)
       .toList();
   final gap = layout.d(
     tall ? FridgeProductTokens.gapPhotos : FridgeProductTokens.gapSmallPhotos,
   );
   final columnWidth = math.max(0.0, (width - 2 * gap) / 3);
   final diameter = math.min(
     columnWidth,
     layout.d(FridgeProductTokens.sizeHomePhoto),
   );
   final style = TextStyle(
     fontSize: layout.font(FridgeProductTokens.fontPhotoNumber),
     height: 1.22,
     color: palette.color('caption'),
   );
   var captionHeight = layout.font(FridgeProductTokens.fontPhotoNumber) * 1.22;
   for (final label in [
     ...shown.map((food) => '#${food.id}'),
```

```
      if (foods.length > capacity) strings.text('more'),
    ]) {
      final painter = TextPainter(
        text: TextSpan(text: label, style: style),
        textDirection: Directionality.of(context),
        textScaler: MediaQuery.textScalerOf(context),
      )..layout(maxWidth: columnWidth);
      captionHeight = math.max(captionHeight, painter.height);
    }
    final rowHeight =
        diameter +
        layout.d(FridgeProductTokens.spacePhotoCaptionTop) +
        captionHeight;
    final slotHeight = tall
        ? math.max(rowHeight, layout.d(FridgeProductTokens.sizeHomeSlot))
        : rowHeight;
    final gridHeight = math.max(
      layout.d(
        tall
            ? FridgeProductTokens.sizeHomeGrid
            : FridgeProductTokens.sizeSmallGrid,
      ),
      tall
          ? slotHeight * 2 + layout.d(FridgeProductTokens.gapPhotoRows)
          : slotHeight,
    );
    Widget slot(Widget circle, String label) => SizedBox(
      width: columnWidth,
      height: slotHeight,
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          circle,
          SizedBox(height: layout.d(FridgeProductTokens.spacePhotoCaptionTop)),
          Text(label, style: style, textAlign: TextAlign.center),
        ],
      ),
    );
    return SizedBox(
      height: gridHeight,
      child: foods.isEmpty
          ? Center(
              child: Text(
                '+',
                style: TextStyle(
                  fontSize: layout.font(FridgeProductTokens.fontEmptyPlus),
                  color: palette.muted,
                  height: 1,
                ),
              ),
            )
          : Align(
              alignment: AlignmentDirectional.topStart,
              child: Wrap(
                spacing: gap,
                runSpacing: layout.d(FridgeProductTokens.gapPhotoRows),
                children: [
```

```
                  for (final food in shown)
                    slot(
                      FridgeFoodPhoto(
                        key: ValueKey('home-photo-${food.id}'),
                        image: photo(food.photoPath),
                        size: diameter,
                        scale: layout.scale,
                        errorLabel: strings.text('error'),
                      ),
                      '#${food.id}',
                    ),
                  if (foods.length > capacity)
                    slot(
                      FridgeMorePhoto(
                        size: diameter,
                        scale: layout.scale,
                        count: '+${foods.length - shown.length}',
                      ),
                      strings.text('more'),
                    ),
                ],
              ),
            ),
    );
  }
}
part 'capture_controls.dart';
part 'capture_viewfinder.dart';
class CaptureScreen extends StatelessWidget {
  final FridgeStrings strings;
  final StorageZone zone;
  final int savedCount;
  final bool ready, opening, busy, pending, failed;
  final Widget preview;
  final ValueChanged<StorageZone> onZone;
  final VoidCallback onCapture,
      onAlbum,
      onRetry,
      onCameraRetry,
      onSettings,
      onFinish;
  const CaptureScreen({
    super.key,
    required this.strings,
    required this.zone,
    required this.savedCount,
    required this.ready,
    required this.opening,
    required this.busy,
    required this.pending,
    required this.failed,
    required this.preview,
    required this.onZone,
    required this.onCapture,
    required this.onAlbum,
    required this.onRetry,
    required this.onCameraRetry,
```

```
    required this.onSettings,
    required this.onFinish,
  });
  bool get locked => busy || pending;
  @override
  Widget build(BuildContext context) => Theme(
    data: fridgeCameraTheme(),
    child: FridgePage(
      title: strings.text('add'),
      backIcon: FridgeIconName.cameraBack,
      backIconOriginalColors: true,
      titleWeight: FontWeight.w500,
      titleHeight: 1.5,
      headerTop: FridgeProductTokens.spaceCameraHeaderTop,
      child: Builder(
        builder: (context) {
          final l = FridgeLayout.of(context);
          return CustomScrollView(
            slivers: [
              SliverToBoxAdapter(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.stretch,
                  children: [
                    SizedBox(
                      height: l.d(FridgeProductTokens.spaceCameraIntroTop),
                    ),
                    _localTag(context),
                    SizedBox(height: l.d(FridgeProductTokens.spaceRow)),
                    _selector(context),
                    SizedBox(
                      height: l.d(FridgeProductTokens.spaceCameraPreviewTop),
                    ),
                    _viewfinder(context),
                    SizedBox(
                      height: l.d(FridgeProductTokens.spaceCameraControlsTop),
                    ),
                    _controls(context),
                  ],
                ),
              ),
              _footer(context),
            ],
          );
        },
      ),
    ),
  );
}
part of 'capture_screen.dart';
extension _CaptureViewfinder on CaptureScreen {
  Widget _viewfinder(BuildContext context) {
    final l = FridgeLayout.of(context);
    final p = Theme.of(context).extension<FridgePalette>()!;
    return AspectRatio(
      aspectRatio: 1,
      child: Container(
        key: const ValueKey('capture-preview'),
```

```
        clipBehavior: Clip.antiAlias,
        decoration: BoxDecoration(
          color: p.color('cameraControl'),
          borderRadius: BorderRadius.circular(
            l.d(FridgeProductTokens.radiusCameraPreview),
          ),
          border: Border.all(
            color: p.color('cameraPreviewBorder'),
            width: l.d(FridgeProductTokens.sizeBorder),
          ),
        ),
        child: Stack(
          fit: StackFit.expand,
          children: [
            if (ready) preview,
            if (ready)
              IgnorePointer(
                child: Padding(
                  padding: EdgeInsets.all(
                    l.d(FridgeProductTokens.spaceCameraGuide),
                  ),
                  child: Container(
                    key: const ValueKey('capture-guide'),
                    padding: EdgeInsets.fromLTRB(
                      l.d(FridgeProductTokens.spaceSmall),
                      0,
                      l.d(FridgeProductTokens.spaceSmall),
                      l.d(FridgeProductTokens.spaceCameraGuideBottom),
                    ),
                    decoration: BoxDecoration(
                      borderRadius: BorderRadius.circular(
                        l.d(FridgeProductTokens.radiusOuter),
                      ),
                      border: Border.all(
                        color: p.color('cameraGuideBorder'),
                        width: l.d(FridgeProductTokens.sizeBorder),
                      ),
                    ),
                    alignment: Alignment.bottomCenter,
                    child: Container(
                      padding: EdgeInsets.symmetric(
                        horizontal: l.d(
                          FridgeProductTokens.spaceCameraGuideHorizontal,
                        ),
                        vertical: l.d(
                          FridgeProductTokens.spaceCameraGuideVertical,
                        ),
                      ),
                      decoration: BoxDecoration(
                        color: p.color('cameraGuideScrim'),
                        borderRadius: BorderRadius.circular(
                          l.d(FridgeProductTokens.radiusCameraGuideCaption),
                        ),
                      ),
                      child: Text(
                        strings.text('cameraSingleCaption'),
                        textAlign: TextAlign.center,
```

```
                        style: TextStyle(
                          fontSize: l.font(FridgeProductTokens.fontMetadata),
                          height: 1.4,
                          color: p.color('cameraGuideInk'),
                        ),
                      ),
                    ),
                  ),
                ),
              ),
            ),
            if (!ready)
              Padding(
                padding: EdgeInsets.all(l.d(FridgeProductTokens.spacePage)),
                child: Center(
                  child: opening
                      ? const CircularProgressIndicator()
                      : SingleChildScrollView(
                          child: Column(
                            mainAxisSize: MainAxisSize.min,
                            children: [
                              Text(
                                strings.text('cameraUnavailable'),
                                textAlign: TextAlign.center,
                              ),
                              TextButton(
                                onPressed: onCameraRetry,
                                child: Text(strings.text('retry')),
                              ),
                              TextButton(
                                onPressed: onSettings,
                                child: Text(strings.text('openSettings')),
                              ),
                            ],
                          ),
                        ),
                ),
              ),
          ],
        ),
      ),
    );
  }
}
part of 'record_editor.dart';
extension _RecordFields on _RecordEditorState {
  Widget _fields(BuildContext context) {
    final layout = FridgeLayout.of(context);
    final palette = Theme.of(context).extension<FridgePalette>()!;
    final labelStyle = TextStyle(
      fontSize: layout.font(FridgeProductTokens.fontSecondary),
      height: 4 / 3,
      color: palette.color('editorLabel'),
    );
    InputDecoration decoration() => InputDecoration(
      isDense: true,
      filled: true,
      fillColor: palette.color('surface'),
```

```
      contentPadding: EdgeInsets.all(
        layout.d(FridgeProductTokens.spaceFieldPadding),
      ),
      constraints: const BoxConstraints(minHeight: 44),
      border: OutlineInputBorder(
        borderRadius: BorderRadius.circular(
          layout.d(FridgeProductTokens.radiusChoice),
        ),
        borderSide: BorderSide(color: palette.border, width: layout.d(.8)),
      ),
      enabledBorder: OutlineInputBorder(
        borderRadius: BorderRadius.circular(
          layout.d(FridgeProductTokens.radiusChoice),
        ),
        borderSide: BorderSide(color: palette.border, width: layout.d(.8)),
      ),
    );
    Widget field(String key, Widget child) => Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        Text(strings.text(key), style: labelStyle),
        SizedBox(height: layout.d(FridgeProductTokens.spaceSmall)),
        child,
      ],
    );
    Widget dateField(String key, DateTime value, bool reminder) => field(
      key,
      OutlinedButton(
        key: ValueKey(key),
        onPressed: locked ? null : () => _pick(reminder),
        style: OutlinedButton.styleFrom(
          backgroundColor: palette.color('surface'),
          foregroundColor: palette.color('ink'),
          minimumSize: const Size(0, 44),
          tapTargetSize: MaterialTapTargetSize.shrinkWrap,
          padding: EdgeInsets.all(
            layout.d(FridgeProductTokens.spaceFieldPadding),
          ),
          side: BorderSide(color: palette.border, width: layout.d(.8)),
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(
              layout.d(FridgeProductTokens.radiusChoice),
            ),
          ),
          textStyle: TextStyle(
            fontSize: layout.font(FridgeProductTokens.fontBody),
            height: 1.3,
            letterSpacing: 0,
            fontWeight: FontWeight.w400,
          ),
        ),
        child: Row(
          children: [
            Expanded(
              child: Text(FridgeDates.editable(context, strings.locale, value)),
            ),
            SizedBox(width: layout.d(FridgeProductTokens.spaceSmall)),
```

```
          SizedBox(
            width: layout.d(FridgeProductTokens.spaceEditorRow),
            height: layout.d(FridgeProductTokens.spaceEditorRow),
            child: Center(
              child: FridgeIcon(FridgeIconName.calendar, scale: layout.scale),
            ),
          ),
        ],
      ),
    ),
  );
  final zoneField = field(
    'storageZone',
    DropdownButtonFormField<StorageZone>(
      key: const ValueKey('record-zone'),
      initialValue: zone,
      decoration: decoration().copyWith(
        contentPadding: EdgeInsetsDirectional.fromSTEB(
          layout.d(FridgeProductTokens.spaceFieldPadding),
          layout.d(FridgeProductTokens.spaceIntroTop),
          layout.d(FridgeProductTokens.spaceEditorChevronEnd),
          layout.d(FridgeProductTokens.spaceIntroTop),
        ),
      ),
      isExpanded: true,
      style: TextStyle(
        fontSize: layout.font(FridgeProductTokens.fontSecondary),
        color: palette.color('editorLabel'),
        height: 1.3,
        letterSpacing: 0,
      ),
      iconSize: layout.d(FridgeProductTokens.fontCaption),
      icon: Transform.translate(
        offset: Offset(
          layout.d(FridgeProductTokens.spaceEditorChevronOffset) *
              (Directionality.of(context) == TextDirection.rtl ? -1 : 1),
          0,
        ),
        child: Center(
          widthFactor: 1,
          heightFactor: 1,
          child: FridgeIcon(
            FridgeIconName.editorChevron,
            scale: layout.scale,
          ),
        ),
      ),
      items: StorageZone.values
          .map(
            (z) =>
                DropdownMenuItem(value: z, child: Text(strings.text(z.name))),
          )
          .toList(),
      onChanged: locked
          ? null
          : (value) {
              if (value != null) changeZone(value);
```

```
            },
      ),
    );
    final intervalField = field(
      'interval',
      Semantics(
        label: strings.text('interval'),
        child: Focus(
          onFocusChange: changeIntervalFocus,
          child: Container(
            key: const ValueKey('record-interval-surface'),
            alignment: Alignment.center,
            constraints: const BoxConstraints(minHeight: 44),
            padding: EdgeInsets.all(
              layout.d(FridgeProductTokens.spaceFieldPadding),
            ),
            decoration: BoxDecoration(
              color: palette.color('surface'),
              borderRadius: BorderRadius.circular(
                layout.d(FridgeProductTokens.radiusChoice),
              ),
              border: Border.all(
                color: palette.color(intervalFocused ? 'primary' : 'line'),
                width: layout.d(FridgeProductTokens.sizeBorder),
              ),
            ),
            child: TextField(
              key: const ValueKey('record-interval'),
              controller: interval,
              enabled: !locked,
              keyboardType: TextInputType.number,
              style: TextStyle(
                fontSize: layout.font(FridgeProductTokens.fontSecondary),
                color: palette.color('editorLabel'),
                height: 1.3,
                letterSpacing: 0,
              ),
              decoration: const InputDecoration(
                isCollapsed: true,
                isDense: true,
                filled: false,
                border: InputBorder.none,
                enabledBorder: InputBorder.none,
                focusedBorder: InputBorder.none,
                disabledBorder: InputBorder.none,
                contentPadding: EdgeInsets.zero,
              ),
            ),
          ),
        ),
      ),
    );
    return Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        dateField('addedAt', addedAt, false),
        SizedBox(height: layout.d(FridgeProductTokens.spaceEditorRow)),
```

```
        LayoutBuilder(
          builder: (context, constraints) {
            if (constraints.maxWidth < 260 ||
                MediaQuery.textScalerOf(context).scale(12) > 16) {
              return Column(
                crossAxisAlignment: CrossAxisAlignment.stretch,
                children: [
                  zoneField,
                  SizedBox(
                    height: layout.d(FridgeProductTokens.spaceEditorRow),
                  ),
                  intervalField,
                ],
              );
            }
            return Row(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Expanded(child: zoneField),
                SizedBox(width: layout.d(FridgeProductTokens.spaceRow)),
                Expanded(child: intervalField),
              ],
            );
          },
        ),
        SizedBox(height: layout.d(FridgeProductTokens.spaceEditorRow)),
        dateField('nextCheck', dueAt, true),
      ],
    );
  }
}
enum DeletionPhase { prepared, submitting, unknown, accepted, cancelled }
DateTime? deletionInstant(String value) {
  final match = RegExp(
    r'^(\d{4})-(\d{2})-(\d{2})T(\d{2}):(\d{2}):(\d{2})(?:\.(\d{1,9}))?Z$',
  ).firstMatch(value);
  if (match == null) return null;
  final parts = List.generate(6, (i) => int.parse(match.group(i + 1)!));
  final date = DateTime.tryParse(value);
  if (date == null ||
      !listEquals(parts, [
        date.year,
        date.month,
        date.day,
        date.hour,
        date.minute,
        date.second,
      ])) {
    return null;
  }
  return date;
}
class DeletionReceipt {
  final String id, requestedAt, eligibleAt;
  const DeletionReceipt(this.id, this.requestedAt, this.eligibleAt);
  bool get valid {
    final requested = deletionInstant(requestedAt);
```

```
    final eligible = deletionInstant(eligibleAt);
    return RegExp(r'^[A-Za-z0-9][A-Za-z0-9._:-]{0,127}$').hasMatch(id) &&
        requested != null &&
        eligible != null &&
        eligible.difference(requested) >= const Duration(days: 7);
  }
}
class DeletionEntry {
  final String owner, key;
  final DeletionPhase phase;
  final DeletionReceipt? receipt;
  const DeletionEntry(this.owner, this.key, this.phase, [this.receipt]);
  DeletionEntry withPhase(DeletionPhase next) =>
      DeletionEntry(owner, key, next, receipt);
}
abstract interface class DeletionStorage {
  Future<List<DeletionEntry>> load();
  Future<void> save(List<DeletionEntry> entries);
}
abstract interface class DeletionContext {
  String get owner;
  bool get current;
  Future<String> challenge(String key);
  Future<String> authorize();
  Future<String> confirm(String challenge, String proof, String key);
  Future<void> stopReminders();
  Future<DeletionReceipt> submit(String key, String confirmation);
  Future<void> finish();
  void cancel();
}
abstract interface class DeletionGateway {
  Future<DeletionContext> open();
}
class DeletionFailure implements Exception {
  final String key;
  const DeletionFailure(this.key);
}
class AccountDeletionSession extends ChangeNotifier {
  final DeletionStorage storage;
  final DeletionGateway gateway;
  final bool Function() consent;
  final String Function() nextKey;
  List<DeletionEntry> _entries = [];
  DeletionEntry? entry;
  bool loading = true, initialized = false, busy = false;
  bool durable = true, cleanupPending = false;
  String? errorKey;
  bool _closed = false;
  int _generation = 0;
  Future<void>? _starting, _operation;
  DeletionContext? _context;
  Future<void> Function()? _pendingFinish;
  AccountDeletionSession({
    required this.storage,
    required this.gateway,
    required this.consent,
    String Function()? nextKey,
```

```
  }) : nextKey =
           nextKey ??
           (() =>
               'fridge-delete-${DateTime.now().microsecondsSinceEpoch}-
${Random.secure().nextInt(0x7fffffff).toRadixString(16)}');
  bool get receiptIncomplete =>
      entry?.phase == DeletionPhase.accepted && (!durable || cleanupPending);
  void _notify() {
    if (!_closed) notifyListeners();
  }
  void _gate(int generation, [DeletionContext? context]) {
    if (_closed ||
        !consent() ||
        generation != _generation ||
        context != null && !context.current) {
      throw const DeletionFailure('deletionCancelled');
    }
  }
  Future<void> start() {
    if (_closed || initialized) return Future.value();
    if (_starting case final starting?) return starting;
    final generation = _generation;
    loading = true;
    return _starting = Future<void>(() async {
      try {
        _gate(generation);
        final loaded = await storage.load();
        _gate(generation);
        _entries = loaded
            .map(
              (value) => value.phase == DeletionPhase.submitting
                  ? value.withPhase(DeletionPhase.unknown)
                  : value,
            )
            .toList();
        entry = _entries.lastOrNull;
        initialized = true;
        errorKey = null;
      } catch (error) {
        errorKey = error is DeletionFailure
            ? error.key
            : 'deletionStorageFailed';
      } finally {
        loading = false;
        _notify();
      }
    }).whenComplete(() => _starting = null);
  }
  Future<void> _save(DeletionEntry value, int generation) async {
    _gate(generation);
    final next = [
      ..._entries.where((item) => item.owner != value.owner),
      value,
    ];
    try {
      await storage.save(List.unmodifiable(next));
    } catch (_) {
```

```
      throw const DeletionFailure('deletionStorageFailed');
    }
    _gate(generation);
    _entries = next;
    entry = value;
    durable = true;
    _notify();
  }
  Future<void> _run(Future<void> Function(int) action) {
    if (_closed) return Future.value();
    if (_operation case final operation?) return operation;
    final generation = _generation;
    busy = true;
    errorKey = null;
    final operation = Future<void>(() async {
      try {
        await start();
        _gate(generation);
        if (!initialized) throw const DeletionFailure('deletionStorageFailed');
        await action(generation);
      } catch (error) {
        errorKey = error is DeletionFailure ? error.key : 'deletionFailed';
      } finally {
        _context?.cancel();
        _context = null;
        busy = false;
        _notify();
      }
    }).whenComplete(() => _operation = null);
    _operation = operation;
    _notify();
    return operation;
  }
  Future<void> submit(Future<bool> Function() confirmVisible) {
    if (receiptIncomplete) return retryReceipt();
    return _run((generation) async {
      final context = await gateway.open();
      _context = context;
      _gate(generation, context);
      var original = _entries
          .where((e) => e.owner == context.owner)
          .firstOrNull;
      if (original?.phase == DeletionPhase.accepted) {
        original = original!.withPhase(DeletionPhase.cancelled);
        await _save(original, generation);
      }
      entry = original;
      durable = true;
      _notify();
      if (!await confirmVisible()) return;
      _gate(generation, context);
      final key = original != null && original.phase != DeletionPhase.cancelled
          ? original.key
          : nextKey();
      var pending = DeletionEntry(
        context.owner,
        key,
```

```
        original?.phase == DeletionPhase.unknown
            ? DeletionPhase.unknown
            : DeletionPhase.prepared,
      );
      await _save(pending, generation);
      _gate(generation, context);
      final challenge = await context.challenge(key);
      _gate(generation, context);
      final proof = await context.authorize();
      _gate(generation, context);
      final confirmation = await context.confirm(challenge, proof, key);
      _gate(generation, context);
      await context.stopReminders();
      _gate(generation, context);
      pending = pending.withPhase(DeletionPhase.submitting);
      await _save(pending, generation);
      _gate(generation, context);
      DeletionReceipt receipt;
      try {
        receipt = await context.submit(key, confirmation);
        _gate(generation);
        if (!receipt.valid) throw const DeletionFailure('deletionUnknown');
      } catch (error) {
        pending = pending.withPhase(DeletionPhase.unknown);
        entry = pending;
        if (!_closed && consent() && generation == _generation) {
          await _save(pending, generation);
        }
        throw error is DeletionFailure && error.key == 'deletionCancelled'
            ? error
            : const DeletionFailure('deletionUnknown');
      }
      pending = DeletionEntry(
        context.owner,
        key,
        DeletionPhase.accepted,
        receipt,
      );
      entry = pending;
      durable = false;
      cleanupPending = true;
      _pendingFinish = context.finish;
      _notify();
      await _save(pending, generation);
      _gate(generation);
      await _finish();
    });
  }
  Future<void> retryReceipt() => _run((generation) async {
    final value = entry;
    if (value == null || value.phase != DeletionPhase.accepted) return;
    await _save(value, generation);
    _gate(generation);
    await _finish();
  });
  Future<void> refreshOwner({bool afterLogin = false}) {
    if (afterLogin && _operation != null) {
```

```
      return _operation!.then((_) => refreshOwner(afterLogin: true));
    }
    return _run((generation) async {
      final context = await gateway.open();
      _context = context;
      _gate(generation, context);
      final value = _entries.where((e) => e.owner == context.owner).firstOrNull;
      if (value != null &&
          (value.phase == DeletionPhase.accepted ||
              afterLogin &&
                  [
                    DeletionPhase.unknown,
                    DeletionPhase.submitting,
                  ].contains(value.phase))) {
        await _save(value.withPhase(DeletionPhase.cancelled), generation);
      } else {
        entry = value;
        durable = true;
        _notify();
      }
    });
  }
  Future<void> _finish() async {
    try {
      await _pendingFinish?.call();
      _pendingFinish = null;
      cleanupPending = false;
      _notify();
    } catch (_) {
      throw const DeletionFailure('deletionCleanupFailed');
    }
  }
  void freeze() {
    ++_generation;
    _context?.cancel();
  }
  Future<void> close() async {
    _closed = true;
    freeze();
    _pendingFinish = null;
    await _starting;
    await _operation;
  }
  @override
  void dispose() {
    unawaited(close());
    super.dispose();
  }
}
typedef FeedbackDraft = ({String text, String? key, String? receipt});
abstract interface class FeedbackDraftStorage {
  Future<FeedbackDraft> load();
  Future<void> save(FeedbackDraft value);
}
abstract interface class FeedbackGateway {
  Future<String> submit(String message, String idempotencyKey);
}
```

```
class FeedbackFailure implements Exception {
  final String key;
  const FeedbackFailure(this.key);
}
class FeedbackSession extends ChangeNotifier {
  final FeedbackDraftStorage storage;
  final FeedbackGateway gateway;
  final bool Function() consent;
  final String Function() nextKey;
  final ProductEvent? onEvent;
  String text = '';
  String? requestKey, receipt, errorKey;
  bool loading = true, busy = false, initialized = false, _closed = false;
  Future<void>? _starting, _submission;
  Future<void> _writes = Future.value();
  int _editRevision = 0;
  FeedbackSession({
    required this.storage,
    required this.gateway,
    required this.consent,
    String Function()? nextKey,
    this.onEvent,
  }) : nextKey =
           nextKey ??
           () =>
               'fridge-feedback-${DateTime.now().microsecondsSinceEpoch}-
${Random.secure().nextInt(0x7fffffff).toRadixString(16)}');
  void _notify() {
    if (!_closed) notifyListeners();
  }
  Future<void> start() {
    if (_closed || initialized) return Future.value();
    if (_starting case final operation?) return operation;
    loading = true;
    errorKey = null;
    final operation = Future<void>(() async {
      try {
        final value = await storage.load();
        text = value.text;
        requestKey = value.key;
        receipt = value.receipt;
        initialized = true;
      } catch (_) {
        errorKey = 'feedbackDraftLoadFailed';
      } finally {
        loading = false;
        _notify();
      }
    });
    _starting = operation;
    _notify();
    return operation.whenComplete(() => _starting = null);
  }
  Future<void> _persist(FeedbackDraft value) {
    final operation = _writes.then((_) => storage.save(value));
    _writes = operation.catchError((Object _) {});
    return operation;
```

```
  }
  Future<void> edit(String value) async {
    if (_closed || !initialized || busy || receipt != null || value == text) {
      return;
    }
    final samePayload = text.trim() == value.trim();
    text = value;
    if (!samePayload) requestKey = null;
    final revision = ++_editRevision;
    errorKey = null;
    _notify();
    try {
      await _persist((text: text, key: requestKey, receipt: null));
    } catch (_) {
      if (revision == _editRevision && !busy && receipt == null) {
        errorKey = 'feedbackDraftSaveFailed';
        _notify();
      }
    }
  }
  Future<void> submit() {
    if (_closed || !initialized || receipt != null) return Future.value();
    if (busy) return _submission ?? Future.value();
    final message = text.trim();
    if (message.isEmpty || message.length > 4000) {
      errorKey = 'feedbackValidation';
      _notify();
      return Future.value();
    }
    busy = true;
    errorKey = null;
    final operation = Future<void>(() async {
      try {
        if (!consent()) throw const FeedbackFailure('feedbackUnavailable');
        requestKey ??= nextKey();
        try {
          await _persist((text: text, key: requestKey, receipt: null));
        } catch (_) {
          throw const FeedbackFailure('feedbackDraftSaveFailed');
        }
        if (!consent()) throw const FeedbackFailure('feedbackUnavailable');
        final accepted = await gateway.submit(message, requestKey!);
        if (!consent()) throw const FeedbackFailure('feedbackUnavailable');
        if (accepted.trim().isEmpty || accepted.length > 128) {
          throw const FeedbackFailure('feedbackFailed');
        }
        try {
          await _persist((text: '', key: null, receipt: accepted));
        } catch (_) {
          throw const FeedbackFailure('feedbackDraftSaveFailed');
        }
        text = '';
        requestKey = null;
        receipt = accepted;
        errorKey = null;
        _event('success');
      } on FeedbackFailure catch (failure) {
```

```
        errorKey = failure.key;
        _event(
          failure.key.startsWith('feedbackDraft')
              ? 'storage_failed'
              : failure.key == 'feedbackUnavailable' ||
                    failure.key == 'feedbackSessionUnavailable'
              ? 'identity_unavailable'
              : 'submit_failed',
        );
      } catch (_) {
        errorKey = 'feedbackFailed';
        _event('submit_failed');
      } finally {
        busy = false;
        _notify();
      }
    });
    _submission = operation;
    _notify();
    return operation;
  }
  Future<void> writeAnother() async {
    if (_closed || busy || receipt == null) return;
    busy = true;
    errorKey = null;
    _notify();
    try {
      await _persist((text: '', key: null, receipt: null));
      receipt = null;
      text = '';
      requestKey = null;
    } catch (_) {
      errorKey = 'feedbackDraftSaveFailed';
    } finally {
      busy = false;
      _notify();
    }
  }
  void _event(String result) {
    if (!_closed && consent()) {
      emitProductEvent(onEvent, 'fridge.feedback.submitted.v1', {
        'result': result,
      });
    }
  }
  Future<void> close() async {
    if (_closed) return;
    _closed = true;
    await _starting;
    await _submission;
    await _writes;
  }
  @override
  void dispose() {
    unawaited(close());
    super.dispose();
  }
```

```
}
part of 'bootstrap.dart';
extension PrivacyActions on FridgeBootstrap {
  Future<bool> openLegal(String key) async {
    if (!legal.containsKey(key)) throw ArgumentError.value(key);
    if (_legalBusy) return false;
    _legalBusy = true;
    final allowWithdrawal = key == 'privacy' && onlineConsent;
    var html = legal[key]!;
    if (appearance != DisplayAppearance.system) {
      html = html.replaceAll(
        '@media(prefers-color-scheme:dark)',
        appearance == DisplayAppearance.dark ? '@media all' : '@media not all',
      );
    }
    if (allowWithdrawal) {
      final label = const HtmlEscape().convert(
        strings!.text('privacyWithdrawLink'),
      );
      html = html.replaceFirst(
        '</main>',
        '<footer><a href="fridge://privacy/withdraw" lang="${const
HtmlEscape().convert(strings!.locale)}" dir="auto" '
            'style="display:block;min-height:44px;margin-top:24px;padding:12px
0;color:inherit;font:inherit;line-height:1.6">$label</a></footer></main>',
      );
    }
    try {
      final result = await native.invokeMethod<Object>('legal.open', {
        'html': html,
        'htmlBase64': base64Encode(utf8.encode(html)),
        'title': strings!.text(key),
        'backLabel': strings!.text('back'),
        'errorLabel': strings!.text('error'),
        'appearance': appearance.name,
        'withdrawalEnabled': allowWithdrawal ? 'true' : 'false',
      });
      if (result == 'withdraw') return allowWithdrawal && onlineConsent;
      if (result != true) throw StateError('LEGAL_OPEN_FAILED');
      return false;
    } finally {
      _legalBusy = false;
    }
  }
  Future<void> withdrawPrivacy() {
    if (_withdrawal case final pending?) return pending;
    _privacyAccepted = false;
    _online.invalidate();
    _busy = true;
    telemetry?.close();
    final deletionClosing = deletion?.close();
    if (deletionClosing != null) unawaited(deletionClosing);
    stage = BootstrapStage.loading;
    localReminders?.setActive(false);
    final draftClosing = feedback?.close();
    if (draftClosing != null) unawaited(draftClosing);
    _withdrawal =
```

```
        Future<void>(() async {
          var failed = false;
          try {
            await identity?.stopForWithdrawal();
          } catch (_) {
            failed = true;
          }
          try {
            if (await native.invokeMethod<bool>('privacy.revoke') != true) {
              failed = true;
            }
          } catch (_) {
            failed = true;
          }
          try {
            final path = await native.invokeMethod<String>('privacy.directory');
            if (path == null || path.isEmpty) {
              throw StateError('PRIVACY_CLEANUP_UNAVAILABLE');
            }
            await IdentityStorage.clearRevokedSession(Directory(path));
          } catch (_) {
            failed = true;
          }
          try {
            await localReminders?.close();
          } catch (_) {
            failed = true;
          }
          try {
            await inventory?.close();
          } catch (_) {
            failed = true;
          }
          if (!failed) {
            try {
              if (await native.invokeMethod<bool>('privacy.finishRevoke') !=
                  true) {
                failed = true;
              }
            } catch (_) {
              failed = true;
            }
          }
          if (failed) {
            stage = BootstrapStage.withdrawalError;
            return;
          }
          localReminders?.dispose();
          localReminders = null;
          feedback?.dispose();
          feedback = null;
          deletion?.dispose();
          deletion = null;
          identity?.dispose();
          identity = null;
          telemetry = null;
          inventory = null;
```

```
          dataDirectory = null;
          if (_disposed) return;
          if (FridgeBootstrap.channel == 'global') {
            try {
              await _startBusiness();
            } catch (_) {
              stage = BootstrapStage.error;
            }
          } else {
            stage = BootstrapStage.consent;
          }
        }).whenComplete(() {
          _withdrawal = null;
          _busy = false;
          notifyListeners();
        });
    notifyListeners();
    return _withdrawal!;
  }
}
class FileFeedbackDraftStorage implements FeedbackDraftStorage {
  final Directory directory;
  FileFeedbackDraftStorage(this.directory);
  File get _file => File('${directory.path}/feedback-draft.json');
  @override
  Future<FeedbackDraft> load() async {
    if (!await _file.exists()) return (text: '', key: null, receipt: null);
    try {
      final value = jsonDecode(await _file.readAsString());
      if (value is! Map ||
          value['text'] is! String ||
          (value['key'] != null && value['key'] is! String) ||
          (value['receipt'] != null && value['receipt'] is! String)) {
        throw const FormatException('FEEDBACK_DRAFT_INVALID');
      }
      return (
        text: value['text'] as String,
        key: value['key'] as String?,
        receipt: value['receipt'] as String?,
      );
    } on FormatException {
      await _file.copy(
        '${_file.path}.corrupt-${DateTime.now().microsecondsSinceEpoch}',
      );
      rethrow;
    }
  }
  @override
  Future<void> save(FeedbackDraft value) async {
    await directory.create(recursive: true);
    final pending = File('${_file.path}.pending');
    await pending.writeAsString(
      jsonEncode({
        'text': value.text,
        'key': value.key,
        'receipt': value.receipt,
      }),
```

```
      flush: true,
    );
    await pending.rename(_file.path);
  }
}
String _photoKey(String path) {
  final uri = File(path).absolute.uri.normalizePath().toString();
  return Platform.isWindows ? uri.toLowerCase() : uri;
}
class CameraSession extends ChangeNotifier {
  CameraController? controller;
  bool opening = false;
  bool failed = false;
  bool _closed = false;
  int _generation = 0;
  bool _resumeRequested = false;
  final Future<bool> Function() authorize;
  CameraSession(this.authorize);
  Future<void> open() async {
    if (_closed || controller?.value.isInitialized == true) return;
    if (opening) {
      _resumeRequested = true;
      return;
    }
    opening = true;
    failed = false;
    final generation = ++_generation;
    notifyListeners();
    CameraController? candidate;
    try {
      if (!await authorize()) {
        failed = true;
        return;
      }
      if (_closed || generation != _generation) return;
      final cameras = await availableCameras();
      if (cameras.isEmpty) throw StateError('CAMERA_UNAVAILABLE');
      final camera = cameras.firstWhere(
        (c) => c.lensDirection == CameraLensDirection.back,
        orElse: () => cameras.first,
      );
      candidate = CameraController(
        camera,
        ResolutionPreset.high,
        enableAudio: false,
      );
      await candidate.initialize();
      if (_closed || generation != _generation) {
        await candidate.dispose();
        return;
      }
      controller = candidate;
    } catch (_) {
      await candidate?.dispose();
      failed = true;
    } finally {
      opening = false;
```

```
      if (!_closed) notifyListeners();
      final resume = _resumeRequested;
      _resumeRequested = false;
      if (resume && !_closed) unawaited(open());
    }
  }
  Future<void> suspend() async {
    _generation++;
    final previous = controller;
    controller = null;
    await previous?.dispose();
    if (!_closed) notifyListeners();
  }
  Future<void> close() async {
    _closed = true;
    await suspend();
    super.dispose();
  }
}
class ManagedFoodPhotos implements FoodPhotoSource, PhotoLibrary {
  final Directory directory;
  final CameraSession? camera;
  final ImagePicker picker;
  ManagedFoodPhotos({required this.directory, this.camera, ImagePicker? picker})
    : picker = picker ?? ImagePicker();
  @override
  Future<String> persist(String path) async {
    final photos = Directory('${directory.path}/photos');
    await photos.create(recursive: true);
    final random = Random.secure().nextInt(0x7fffffff).toRadixString(16);
    final name = '${DateTime.now().microsecondsSinceEpoch}-$random';
    final temporary = File('${photos.path}/$name.pending');
    final destination = File('${photos.path}/$name.image');
    try {
      await XFile(path).saveTo(temporary.path);
      if (await temporary.length() == 0) throw StateError('PHOTO_EMPTY');
      await temporary.rename(destination.path);
    } catch (_) {
      try {
        if (await temporary.exists()) await temporary.delete();
      } catch (_) {}
      rethrow;
    }
    return destination.path;
  }
  @override
  Future<String> capture() async {
    final controller = camera?.controller;
    if (controller == null || !controller.value.isInitialized) {
      throw StateError('CAMERA_UNAVAILABLE');
    }
    return (await controller.takePicture()).path;
  }
  @override
  Future<List<String>> album() async {
    final selected = await picker.pickMultiImage();
    return selected.map((photo) => photo.path).toList();
```

```
  }
  @override
  Future<String?> replacement() async {
    final selected = await picker.pickImage(source: ImageSource.gallery);
    return selected == null ? null : persist(selected.path);
  }
  @override
  Future<void> collectUnreferenced(Set<String> retained) async {
    final photos = Directory('${directory.path}/photos').absolute;
    if (!await photos.exists()) return;
    final references = retained.map(_photoKey).toSet();
    await for (final file in photos.list(followLinks: false)) {
      if (file is! File) continue;
      final name = file.uri.pathSegments.last;
      if (!RegExp(r'^\d+-[a-f0-9]+\.(image|pending)$').hasMatch(name)) continue;
      if (!references.contains(_photoKey(file.path))) await file.delete();
    }
  }
  @override
  Future<void> discardSaved(String path) async {
    final photo = File(path).absolute;
    final photos = Directory('${directory.path}/photos').absolute;
    if (photo.parent.path != photos.path || !photo.path.endsWith('.image')) {
      throw ArgumentError('UNMANAGED_PHOTO');
    }
    if (await photo.exists()) await photo.delete();
  }
}
class MethodChannelNotifications implements LocalNotificationPlatform {
  final MethodChannel channel;
  final _taps = StreamController<String>.broadcast(sync: true);
  MethodChannelNotifications({
    this.channel = const MethodChannel('fenleng.fridge/notifications'),
  }) {
    channel.setMethodCallHandler((call) async {
      if (call.method == 'openReminder') {
        final args = call.arguments;
        if (args is Map && _taps.hasListener) {
          try {
            final day = await consumeTap();
            if (day != null && !_taps.isClosed) _taps.add(day);
          } catch (_) {}
        }
      }
    });
  }
  @override
  Stream<String> get taps => _taps.stream;
  Future<NotificationStatus> _status(
    String method, [
    Object? arguments,
  ]) async => NotificationStatus.fromJson(
    Map<String, dynamic>.from(
      await channel.invokeMapMethod<String, dynamic>(method, arguments) ?? {},
    ),
  );
  @override
```

```
  Future<NotificationStatus> status() => _status('status');
  @override
  Future<NotificationStatus> request() => _status('request');
  @override
  Future<NotificationStatus> setEnabled(bool value) =>
      _status('enable', {'enabled': value});
  @override
  Future<void> sync(
    List<Map<String, Object>> cycles,
    Map<String, String> copy,
  ) async {
    if (await channel.invokeMethod<bool>('sync', {
          'cycles': cycles,
          'copy': copy,
        }) !=
        true) {
      throw StateError('NOTIFICATION_SYNC_FAILED');
    }
  }
  @override
  Future<String?> consumeTap() => channel.invokeMethod<String>('consumeTap');
  @override
  Future<void> openSettings() async {
    if (await channel.invokeMethod<bool>('settings') != true) {
      throw StateError('NOTIFICATION_SETTINGS_FAILED');
    }
  }
  @override
  Future<void> close() async {
    channel.setMethodCallHandler(null);
    await _taps.close();
  }
}
class ProductTelemetry {
  final bool enabled;
  final bool Function() consent;
  final Future<bool> Function() ensureSession;
  final Duration retryDelay, debounce;
  final Map<String, Map<String, Set<String>>> _catalog;
  late final FlutterTelemetrySDK sdk;
  Timer? _timer;
  Future<int>? _flushing;
  bool _closed = false, _foreground = true;
  int _epoch = 0;
  ProductTelemetry({
    required FenlengFlutterClient client,
    required Map<String, dynamic> catalog,
    required FlutterTelemetryContext Function() context,
    required this.consent,
    required this.ensureSession,
    required this.enabled,
    this.retryDelay = const Duration(milliseconds: 300),
    this.debounce = const Duration(seconds: 2),
  }) : _catalog = _readCatalog(catalog) {
    sdk = FlutterTelemetrySDK(
      client: client,
      hasConsent: () => !_closed && consent(),
```

```
      catalog: _catalog.keys.toSet(),
      context: context,
      deliveryIdFactory: _deliveryId,
      maxQueueSize: 100,
      batchSize: 50,
    );
  }
  int get pendingCount => sdk.pendingCount;
  static String _deliveryId() {
    final random = Random.secure();
    final bytes = List<int>.generate(16, (_) => random.nextInt(256));
    bytes[6] = (bytes[6] & 15) | 64;
    bytes[8] = (bytes[8] & 63) | 128;
    final hex = bytes.map((b) => b.toRadixString(16).padLeft(2, '0')).join();
    return '${hex.substring(0, 8)}-${hex.substring(8, 12)}-${hex.substring(12,
16)}-${hex.substring(16, 20)}-${hex.substring(20)}';
  }
  bool record(String name, [Map<String, String> properties = const {}]) {
    if (_closed || !consent()) return false;
    final allowed = _catalog[name];
    if (allowed == null ||
        properties.length != allowed.length ||
        properties.entries.any(
          (entry) => !(allowed[entry.key]?.contains(entry.value) ?? false),
        )) {
      return false;
    }
    try {
      sdk.track(name, properties);
    } catch (_) {
      return false;
    }
    _schedule();
    return true;
  }
  void _schedule() {
    if (!enabled || _closed || !_foreground || _timer != null || !consent()) {
      return;
    }
    _timer = Timer(debounce, () {
      _timer = null;
      unawaited(flush());
    });
  }
  Future<int> flush() {
    if (_flushing case final pending?) return pending;
    if (!enabled || _closed || !consent() || pendingCount == 0) {
      return Future.value(0);
    }
    final epoch = _epoch;
    _flushing = _drain(epoch).whenComplete(() {
      _flushing = null;
      if (epoch != _epoch && pendingCount > 0) _schedule();
    });
    return _flushing!;
  }
  Future<int> _drain(int epoch) async {
```

```
    for (var attempt = 0; attempt < 3; attempt++) {
      if (_closed || !consent() || epoch != _epoch) return 0;
      try {
        if (!await ensureSession() ||
            _closed ||
            !consent() ||
            epoch != _epoch) {
          return 0;
        }
        final key =
            'fridge-
telemetry-${DateTime.now().microsecondsSinceEpoch}-${Random.secure().nextInt(0x7fffffff)}';
        final accepted = await sdk.flush(key);
        if (epoch == _epoch && pendingCount > 0) _schedule();
        return accepted;
      } catch (_) {
        if (attempt < 2) await Future<void>.delayed(retryDelay * (attempt + 1));
      }
    }
    return 0;
  }
  void resume() {
    _foreground = true;
    if (pendingCount > 0) _schedule();
  }
  Future<void> suspend() async {
    _foreground = false;
    _timer?.cancel();
    _timer = null;
    try {
      await flush().timeout(const Duration(seconds: 2));
    } catch (_) {}
  }
  void withdraw() {
    _epoch++;
    _timer?.cancel();
    _timer = null;
    sdk.withdrawConsent();
  }
  void close() {
    _closed = true;
    withdraw();
  }
}
Map<String, Map<String, Set<String>>> _readCatalog(Map<String, dynamic> input) {
  if (input['separateAnalyticsToggle'] != false ||
      input['consentSource'] != 'unified-privacy-v0.4') {
    throw const FormatException('TELEMETRY_CATALOG_INVALID');
  }
  final result = <String, Map<String, Set<String>>>{};
  for (final event in input['events'] as List) {
    final name = event['name'] as String;
    if (result.containsKey(name)) {
      throw const FormatException('TELEMETRY_CATALOG_INVALID');
    }
    result[name] = (event['properties'] as Map).map(
      (key, values) =>
```

```
          MapEntry(key as String, (values as List).cast<String>().toSet()),
    );
  }
  return result;
}
class AccountDeletionSection extends StatelessWidget {
  final AccountDeletionSession model;
  final FridgeStrings strings;
  final bool signedIn, disabled;
  final VoidCallback onSubmit, onRetry;
  const AccountDeletionSection({
    super.key,
    required this.model,
    required this.strings,
    required this.signedIn,
    required this.disabled,
    required this.onSubmit,
    required this.onRetry,
  });
  @override
  Widget build(BuildContext context) {
    final entry = model.entry, receipt = model.entry?.receipt;
    final accepted = entry?.phase == DeletionPhase.accepted,
        unknown = entry?.phase == DeletionPhase.unknown;
    final error = accepted && !model.durable
        ? 'deletionUnsaved'
        : unknown && model.errorKey == 'deletionUnknown'
        ? null
        : model.errorKey;
    String timestamp(String raw) {
      final value = deletionInstant(raw)!.toLocal();
      final l10n = MaterialLocalizations.of(context);
      return '${l10n.formatFullDate(value)}
${l10n.formatTimeOfDay(TimeOfDay.fromDateTime(value), alwaysUse24HourFormat: true)}';
    }
    return Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        const SizedBox(height: FridgeSpace.page),
        const Divider(),
        const SizedBox(height: FridgeSpace.medium),
        Text(
          strings.text('deletionTitle'),
          style: Theme.of(context).textTheme.titleMedium,
        ),
        const SizedBox(height: FridgeSpace.medium),
        if (model.loading)
          Text(strings.text('loading'))
        else ...[
          if (entry != null)
            Card(
              child: Padding(
                padding: const EdgeInsets.all(FridgeSpace.page),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.stretch,
                  children: [
                    Text(
```

```
                strings.text('deletionLocalRecord'),
                style: Theme.of(context).textTheme.bodySmall,
              ),
              const SizedBox(height: FridgeSpace.small),
              if (accepted) Text(strings.text('deletionPending')),
              if (unknown) Text(strings.text('deletionUnknown')),
              if (entry.phase == DeletionPhase.cancelled)
                Text(
                  strings.text(
                    receipt == null
                        ? 'deletionLoginResolved'
                        : 'deletionRestored',
                  ),
                ),
              if (receipt != null) ...[
                const SizedBox(height: FridgeSpace.medium),
                Text(
                  strings.format('deletionRequestedAt', {
                    'time': timestamp(receipt.requestedAt),
                  }),
                ),
                if (accepted)
                  Text(
                    strings.format('deletionEligibleAt', {
                      'time': timestamp(receipt.eligibleAt),
                    }),
                  ),
              ],
              if (accepted) ...[
                const SizedBox(height: FridgeSpace.medium),
                Text(strings.text('deletionWaitingHint')),
              ],
            ],
          ),
        ),
      ),
    ),
  if (!accepted && !unknown) Text(strings.text('deletionBody')),
  if (error != null) ...[
    const SizedBox(height: FridgeSpace.medium),
    Semantics(
      liveRegion: true,
      child: Text(
        strings.text(error),
        style: TextStyle(color: Theme.of(context).colorScheme.error),
      ),
    ),
  ],
  const SizedBox(height: FridgeSpace.medium),
  if (model.busy)
    Semantics(
      liveRegion: true,
      child: Text(strings.text('deletionProcessing')),
    ),
  if (!model.initialized)
    TextButton(
      onPressed: disabled || model.busy ? null : () => model.start(),
      child: Text(strings.text('retry')),
```

```
            )
          else if (model.receiptIncomplete)
            FilledButton(
              onPressed: disabled || model.busy ? null : onRetry,
              child: Text(strings.text('deletionRetryReceipt')),
            )
          else if (signedIn && !accepted)
            OutlinedButton(
              style: OutlinedButton.styleFrom(
                foregroundColor: Theme.of(context).colorScheme.error,
              ),
              onPressed: disabled || model.busy ? null : onSubmit,
              child: Text(
                strings.text(unknown ? 'deletionRetry' : 'deletionTitle'),
              ),
            )
          else if (!signedIn && !accepted && !unknown)
            Text(strings.text('deletionLoginRequired')),
        ],
      ],
    );
  }
}
class AccountDeletionDialog extends StatefulWidget {
  final FridgeStrings strings;
  final bool china, showApple;
  final IdentityProvider initialProvider;
  final ValueChanged<IdentityProvider> onProvider;
  const AccountDeletionDialog({
    super.key,
    required this.strings,
    required this.china,
    required this.showApple,
    required this.initialProvider,
    required this.onProvider,
  });
  @override
  State<AccountDeletionDialog> createState() => _AccountDeletionDialogState();
}
class _AccountDeletionDialogState extends State<AccountDeletionDialog> {
  late IdentityProvider provider = widget.initialProvider;
  @override
  Widget build(BuildContext context) => Dialog(
    insetPadding: const EdgeInsets.all(FridgeSpace.medium),
    child: ConstrainedBox(
      constraints: const BoxConstraints(maxWidth: 520),
      child: Padding(
        padding: const EdgeInsets.all(FridgeSpace.page),
        child: SingleChildScrollView(
          child: Column(
            mainAxisSize: MainAxisSize.min,
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              Text(
                widget.strings.text('deletionTitle'),
                style: Theme.of(context).textTheme.titleLarge,
              ),
```

```
const SizedBox(height: FridgeSpace.medium),
Text(widget.strings.text('deletionBody')),
const SizedBox(height: FridgeSpace.medium),
Text(widget.strings.text('deletionReauthenticate')),
if (!widget.china || widget.showApple) ...[
  const SizedBox(height: FridgeSpace.medium),
  for (final choice in [
    widget.china
        ? IdentityProvider.wechat
        : IdentityProvider.google,
    if (widget.showApple) IdentityProvider.apple,
  ])
    Padding(
      padding: const EdgeInsets.only(bottom: FridgeSpace.small),
      child: OutlinedButton(
        onPressed: () => setState(() => provider = choice),
        child: Row(
          children: [
            Icon(
              provider == choice
                  ? Icons.radio_button_checked
                  : Icons.radio_button_unchecked,
            ),
            const SizedBox(width: FridgeSpace.small),
            Expanded(
              child: Text(
                widget.strings.text(
                  choice == IdentityProvider.google
                      ? 'googleLogin'
                      : choice == IdentityProvider.wechat
                      ? 'wechatLogin'
                      : 'appleLogin',
                ),
              ),
            ),
          ],
        ),
      ),
    ),
],
const SizedBox(height: FridgeSpace.page),
TextButton(
  autofocus: true,
  onPressed: () => Navigator.pop(context, false),
  child: Text(widget.strings.text('cancel')),
),
const SizedBox(height: FridgeSpace.small),
OutlinedButton(
  style: OutlinedButton.styleFrom(
    foregroundColor: Theme.of(context).colorScheme.error,
  ),
  onPressed: () {
    widget.onProvider(provider);
    Navigator.pop(context, true);
  },
  child: Text(widget.strings.text('deletionConfirmAction')),
),
```

```
            ],
          ),
        ),
      ),
    ),
  );
}
class CapturePage extends StatefulWidget {
  final CameraCoordinator coordinator;
  final FridgeStrings strings;
  const CapturePage({
    super.key,
    required this.coordinator,
    required this.strings,
  });
  @override
  State<CapturePage> createState() => _CapturePageState();
}
class _CapturePageState extends State<CapturePage> with WidgetsBindingObserver {
  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addObserver(this);
    unawaited(widget.coordinator.open());
  }
  @override
  void didChangeAppLifecycleState(AppLifecycleState state) {
    if (state == AppLifecycleState.inactive ||
        state == AppLifecycleState.paused) {
      unawaited(widget.coordinator.camera.suspend());
    } else if (state == AppLifecycleState.resumed) {
      unawaited(widget.coordinator.resume());
    }
  }
  @override
  Widget build(BuildContext context) => Theme(
    data: fridgeCameraTheme(),
    child: ListenableBuilder(
      listenable: widget.coordinator.changes,
      builder: (context, _) {
        final coordinator = widget.coordinator,
            flow = coordinator.flow,
            session = coordinator.camera;
        final strings = widget.strings, controller = session.controller;
        return PopScope(
          canPop: !flow.busy && !flow.hasPending,
          onPopInvokedWithResult: (didPop, _) async {
            if (didPop || flow.busy || !flow.hasPending) return;
            final discard = await showDialog<bool>(
              context: context,
              builder: (context) => AlertDialog(
                content: Text(strings.text('discardCapture')),
                actions: [
                  TextButton(
                    onPressed: () => Navigator.pop(context, false),
                    child: Text(strings.text('cancel')),
                  ),
```

```
                FilledButton(
                  onPressed: () => Navigator.pop(context, true),
                  child: Text(strings.text('done')),
                ),
              ],
            ),
          );
          if (discard == true) {
            await flow.abandon();
            await WidgetsBinding.instance.endOfFrame;
            if (context.mounted) Navigator.pop(context);
          }
        },
        child: CaptureScreen(
          strings: strings,
          zone: flow.zone,
          savedCount: flow.savedCount,
          ready: controller?.value.isInitialized == true,
          opening: session.opening,
          busy: flow.busy,
          pending: flow.hasPending,
          failed: flow.failed,
          preview: controller == null
              ? const SizedBox.shrink()
              : _LiveCamera(controller),
          onZone: flow.selectZone,
          onCapture: flow.take,
          onAlbum: coordinator.chooseAlbum,
          onRetry: flow.retry,
          onCameraRetry: coordinator.open,
          onSettings: coordinator.openSettings,
          onFinish: () => Navigator.of(context).pop(),
        ),
      );
    },
  ),
);
@override
void dispose() {
  WidgetsBinding.instance.removeObserver(this);
  super.dispose();
}
}
class _LiveCamera extends StatelessWidget {
final CameraController controller;
const _LiveCamera(this.controller);
@override
Widget build(BuildContext context) => ValueListenableBuilder<CameraValue>(
  valueListenable: controller,
  builder: (context, value, _) {
    final size = value.previewSize;
    if (!value.isInitialized || size == null) return const SizedBox.shrink();
    final orientation =
        value.lockedCaptureOrientation ?? value.deviceOrientation;
    final portrait =
        orientation == DeviceOrientation.portraitUp ||
        orientation == DeviceOrientation.portraitDown;
```

```
      return FittedBox(
        fit: BoxFit.cover,
        child: SizedBox(
          width: portrait ? size.height : size.width,
          height: portrait ? size.width : size.height,
          child: CameraPreview(controller),
        ),
      );
    },
  );
}
part of '../main.dart';
extension _NotificationRoutes on _FridgeAppState {
  void _attachReminders() {
    if (!mounted) return;
    final model = bootstrap.localReminders;
    if (model != _notifications) {
      _notifications?.removeListener(_openPendingReminder);
      _notifications = model;
      model?.addListener(_openPendingReminder);
    }
    if (model != null) unawaited(model.synchronize());
    _openPendingReminder();
  }
  void _openPendingReminder() {
    if (!mounted ||
        _openingNotification ||
        bootstrap.stage != BootstrapStage.ready ||
        _notifications?.pendingDay == null) {
      return;
    }
    _openingNotification = true;
    WidgetsBinding.instance.addPostFrameCallback((_) async {
      try {
        if (!mounted || bootstrap.stage != BootstrapStage.ready) return;
        final nav = _navigator.currentState;
        if (nav?.overlay == null) return;
        await bootstrap.inventory!.refreshDue();
        if (!mounted || bootstrap.stage != BootstrapStage.ready) return;
        final day = _notifications?.takeTap();
        if (day == null) return;
        final matches = bootstrap.inventory!.state.reminders.where(
          (r) => r.day == day,
        );
        nav!.popUntil((route) => route.isFirst);
        final context = nav.overlay!.context;
        if (!context.mounted) return;
        if (matches.isEmpty) {
          _reminders(context, source: 'notification');
        } else {
          bootstrap.recordEvent('fridge.reminder.opened.v1', {
            'source': 'notification',
          });
          _review(context, reminder: matches.first);
        }
      } catch (_) {
        /* A retained in-app list remains usable on a refresh failure. */
```

```
      }
      finally {
        _openingNotification = false;
      }
    });
    WidgetsBinding.instance.ensureVisualUpdate();
  }
  void _notificationSettings(BuildContext context) {
    final model = bootstrap.localReminders;
    if (model == null) return;
    _push(
      context,
      ListenableBuilder(
        listenable: model,
        builder: (context, _) => NotificationSettingsScreen(
          strings: bootstrap.strings!,
          enabled: model.status.enabled,
          authorized: model.status.authorized,
          requested: model.status.requested,
          busy: model.busy,
          errorKey: model.errorKey,
          limited: model.status.limited,
          timingKey: Platform.isIOS
              ? 'notificationTimingIOS'
              : 'notificationTiming',
          onEnable: () =>
              unawaited(model.enable(() => _explainNotifications(context))),
          onDisable: () => unawaited(model.disable()),
          onSettings: () => unawaited(model.openSettings()),
        ),
      ),
    );
  }
  Future<bool> _explainNotifications(BuildContext context) async {
    if (FridgeBootstrap.channel != 'china') return true;
    if (!context.mounted) return false;
    final strings = bootstrap.strings!;
    return await showDialog<bool>(
          context: context,
          builder: (context) => NotificationPermissionDialog(
            strings: strings,
            onContinue: () => Navigator.pop(context, true),
            onLater: () => Navigator.pop(context, false),
          ),
        ) ??
        false;
  }
}
part of '../main.dart';
extension _PrivacyRoutes on _FridgeAppState {
  Future<bool> _ensureOnline(BuildContext context) async {
    if (bootstrap.onlineConsent) {
      var ready = false;
      await _perform(context, () async {
        await bootstrap.ensureOnlineServices();
        ready = bootstrap.onlineConsent;
      });
```

```
      return ready && context.mounted;
    }
    if (FridgeBootstrap.channel != 'global' ||
        _onlinePromptOpen ||
        !context.mounted) {
      return false;
    }
    _onlinePromptOpen = true;
    try {
      final accepted = await showDialog<bool>(
        context: context,
        barrierDismissible: false,
        builder: (_) => OnlineConsentDialog(
          strings: bootstrap.strings!,
          onAccept: bootstrap.acceptOnline,
          onDocument: (key) => _legal(context, key),
        ),
      );
      return accepted == true && bootstrap.onlineConsent && context.mounted;
    } finally {
      _onlinePromptOpen = false;
    }
  }
  Future<void> _account(BuildContext context) async {
    if (!await _ensureOnline(context) || !context.mounted) return;
    final session = bootstrap.identity;
    if (session == null) return;
    _push(
      context,
      AccountPage(
        session: session,
        strings: bootstrap.strings!,
        china: FridgeBootstrap.channel == 'china',
        showApple: Platform.isIOS,
        deletion: bootstrap.deletion,
        onDelete: bootstrap.deleteAccount,
        onRetryReceipt: bootstrap.retryDeletionReceipt,
        onLogin: bootstrap.loginAccount,
        onLogout: bootstrap.logoutAccount,
        deletionProvider: bootstrap.deletionProvider,
        onDeletionProvider: (provider) => bootstrap.deletionProvider = provider,
      ),
    );
  }
  Future<void> _openLegal(BuildContext context, String key) async {
    if (!await bootstrap.openLegal(key) || !context.mounted) return;
    final confirmed =
        await showDialog<bool>(
          context: context,
          builder: (_) => PrivacyWithdrawalDialog(strings: bootstrap.strings!),
        ) ??
        false;
    if (!confirmed || !mounted) return;
    _navigator.currentState?.popUntil((route) => route.isFirst);
    await bootstrap.withdrawPrivacy();
  }
}
```